M.TECH · CS-4407

Information System Design — Visual Study Book

IS & process models → Project management → Requirements & design → UML → Implementation & testing. Every topic explained with a diagram, a real-life use case, an exam tip and a Hinglish memory hook.

Unit I · Introduction to ISD

Unit II · Project Management

Unit III · Requirements & Design

Unit IV · OOAD & UML

Unit V · Implementation & Testing

70 original diagrams · definitions to open every answer · Hinglish memory hooks · revision checklist

Table of Contents

How to use this book ... 3

Introduction to Information System Design ... 5

1.1 What is an Information System (IS)? ... 5
1.2 Challenges in Information System Design ... 6
1.3 Elements of Information System Design ... 7
1.4 Roles and Responsibilities in IS Design ... 8
1.5 Case Study Approach (IS Design) ... 9
1.6 Software Processes & IS Design Models ... 10

Project Management and Planning ... 15

2.1 Project Management Essentials ... 15
2.2 Project Success and Failure ... 16
2.3 Project Life Cycle ... 16
2.4 Project Team Structure and Organisation ... 17
2.5 Project Planning: Metrics and Measurements ... 18
2.6 Project Estimation — Effort Estimation Techniques ... 19
2.7 Staffing and Personnel Planning ... 21
2.8 Project Scheduling ... 22
2.9 Software Configuration Management (SCM) ... 23
2.10 Risk Management ... 24

Requirements Engineering & Software Design ... 26

3.1 Requirements Engineering (RE) — Overview ... 26
3.2 Requirements Elicitation ... 26
3.3 Requirements Analysis: Structured vs Object-Oriented ... 28
3.4 Requirements Specification (SRS) ... 29
3.5 Requirements Validation ... 30
3.6 Requirements Management and Tools ... 31
3.7 Software Design Fundamentals ... 31
3.8 Design Process ... 33
3.9 Modular Design and Component-Level Design ... 34
3.10 Structured vs Object-Oriented Design ... 34
3.11 Refactoring ... 34

Object-Oriented Analysis, Design & UML Modeling ... 36

4.1 What is UML? ... 36
4.2 Use Case Diagrams ... 37
4.3 Class and Object Diagrams ... 38
4.4 Sequence Diagrams ... 41
4.5 Collaboration / Communication Diagrams ... 42
4.6 State-Chart Diagrams (State Machine) ... 42
4.7 Activity Diagrams ... 43
4.8 Component Diagrams ... 44
4.9 Deployment Diagrams ... 45
4.10 Generalization, Domain Model Refinement, Architecture, Packaging ... 45
4.11 Case Study of Large-Scale Systems (exam approach) ... 47

Implementation and Testing ... 48

5.1 Traditional Implementation vs Test-Driven Development (TDD) ... 48
5.2 Testing of Information Systems ... 49
5.3 Testing Strategies ... 50
5.4 Levels of Testing ... 51
5.5 Debugging ... 52
5.6 Automation Testing ... 53
5.7 Software Testing Tools ... 54

Quick comparison sheet ... 55

Exam-day revision checklist ... 57

Recommended resources ... 58

How to use this book

The 3-step exam answer formula

⭐ **Exam tip — Write every answer in this order**

1. **Definition** — one line (the blue 📘 boxes give you this ready-made).
2. **Explain in 3–5 lines** — plain English, use the keywords in **bold**.
3. **Diagram** — draw the figure from the book. Your notes say it clearly: for UML, practise drawing every diagram 2–3 times by hand.

Box guide

Box	What it gives you
📘 Definition	Opening line for the answer
🏭 Real-life use case	Where this is used in real projects — good for "applications" and viva questions
💡 Why it matters	The reason a concept exists (examiners love "why")
⭐ Exam tip	Marks-scoring points, what to memorise exactly
🧠 Yaad rakho	Short Hinglish memory hook
⚠️ Careful	Common mistakes and traps
❓ Likely exam questions	Past-paper style questions for that unit

Everything comes from your CS-4407 notes. Where this book adds standard textbook material that is **not** in your notes (worked examples, COCOMO constants, sample code), it says so in the text.

Syllabus map

Unit	Outcome	Focus	Main figures to practise
I	CO1, CO2	Information systems, challenges, roles, case-study method, process models	IS cycle · Waterfall · Agile sprint · RUP phases
II	CO3	Project management, estimation, staffing, scheduling, SCM, risk	Iron triangle · COCOMO · PERT · risk cycle
III	CO4	Requirements engineering, SRS, design fundamentals, refactoring	RE cycle · coupling/cohesion · design process
IV	CO5	UML — use case, class, sequence, state, activity, component, deployment	Every UML figure (most diagram-heavy unit)
V	CO4 / CO5	TDD, testing strategies and levels, debugging, automation tools	TDD cycle · testing pyramid · integration approaches

The big picture — how all five units fit together

One information-system project passes through all five units. Keep this picture in mind; each later unit zooms into one stage.

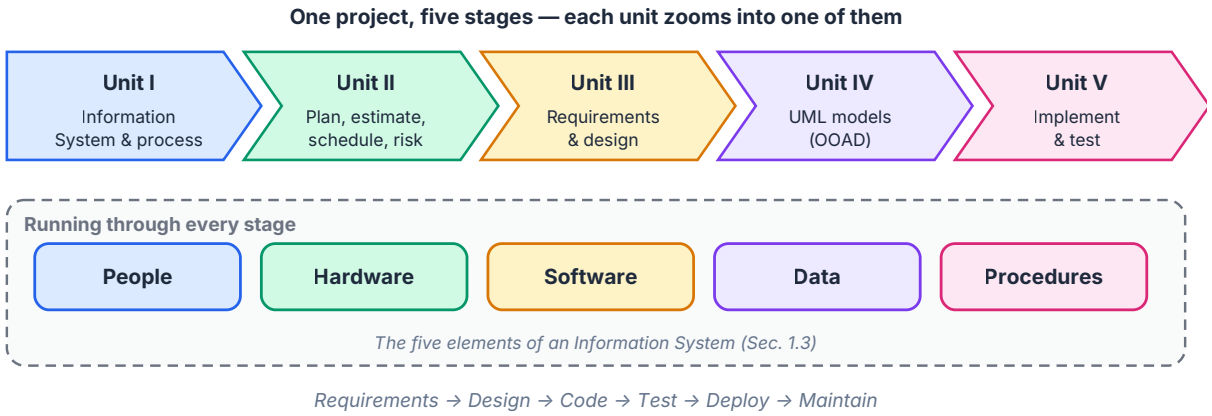


Fig. 1 — One project, five stages — each unit zooms into one of them

🏭 Real-life use case — One project, end to end

A hospital wants a **Patient Management System**. The team first decides what an IS is and which process model fits (Unit I), then plans cost, people, schedule and risks (Unit II). The analyst gathers requirements and the architect designs the structure (Unit III), the team models it in UML (Unit IV), and finally it is coded with tests and debugged before go-live (Unit V).

Practice reading: GeeksforGeeks software-engineering hub — geeksforgeeks.org/software-engineering · OMG UML specification — omg.org/spec/UML · Agile Manifesto — agilemanifesto.org

UNIT I · CO1, CO2

Introduction to Information System Design

What is an IS | Challenges | Elements | Roles | Case study method | Waterfall | CBD | Agile | RUP

1.1 What is an Information System (IS)?

📘 Definition

An **Information System** is an organised combination of **people, hardware, software, communication networks, data resources and policies/procedures** that stores, retrieves, transforms and disseminates information in an organisation.

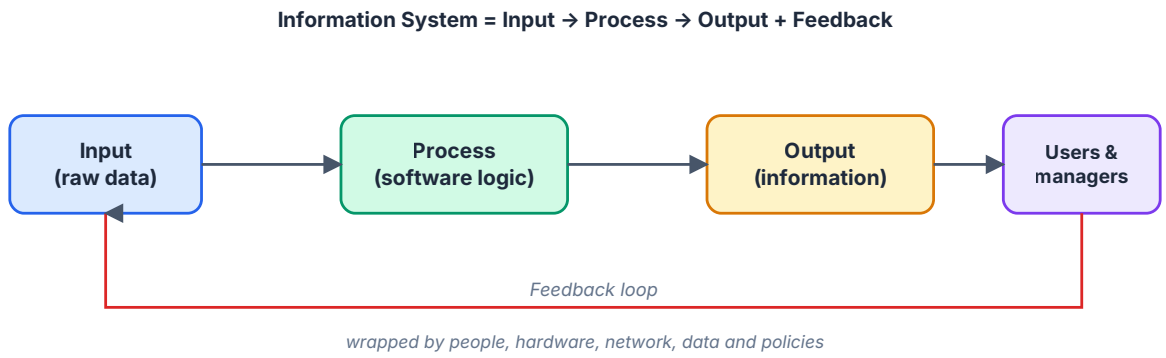


Fig. 2 — Input → Process → Output, with a feedback loop back into the system

Think of it as: **Input** (raw data) → **Process** (software logic) → **Output** (useful information) → **Feedback** → back into the system.

💡 Why it matters

An IS is not just "software". Payroll software without the people who use it, the policies that define salary rules and the hardware that runs it is incomplete. That is why examiners ask you to separate **IS** from **IT**.

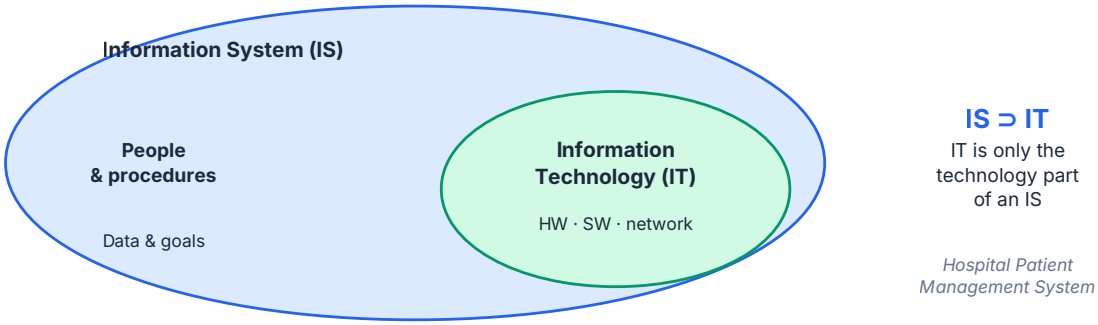


Fig. 3 — IT is only the technology part that sits inside the larger Information System

Point	Information System	Information Technology
Scope	Broader — people + process + technology	Just the technology component (hardware / software / networks)
Nature	Goal-oriented — solves a business problem	Tool-oriented — enables the system
Example	A hospital's Patient Management System	The database server + application code

🏭 Real-life use case — Where you see an IS

A railway reservation system, a college ERP, a bank's core banking system and a hospital patient system are all information systems: each has users, rules, hardware, software and data working together to deliver one business goal.

🧠 Yaad rakho

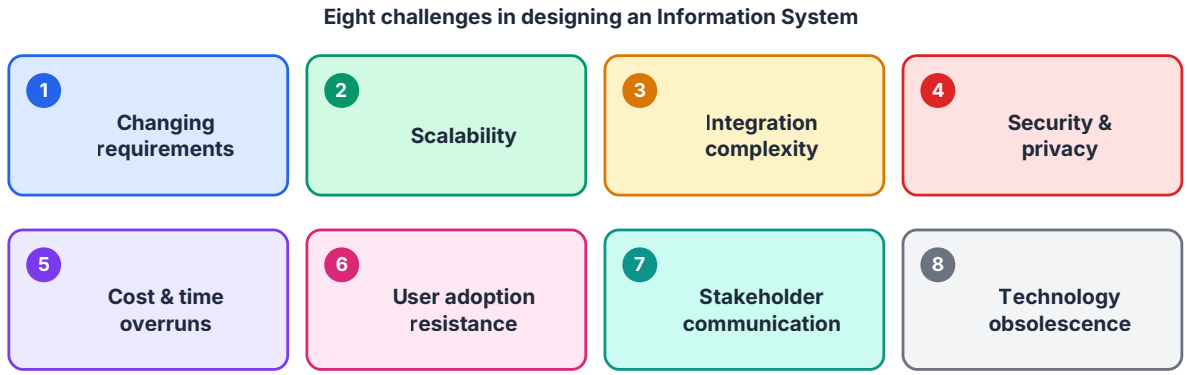
IS = **poora system** (log + rules + tech). IT = sirf **tech wala hissa**. "IT ke andar IS nahi, IS ke andar IT."

📖 GFG: Information System vs Information Technology

1.2 Challenges in Information System Design

📘 Definition

Challenges in ISD are the recurring problems — technical, human and business — that make designing and delivering an information system difficult.



Mnemonic in the notes: C-S-I-S-C-U-T (covers the first 7)

Fig. 4 — Eight challenges in designing an Information System

1. **Changing requirements** — business needs evolve faster than development cycles.
2. **Scalability** — must handle growth in users/data without redesign.
3. **Integration complexity** — legacy systems, third-party APIs, heterogeneous data sources.
4. **Security & privacy** — data breaches, compliance (GDPR-like laws).
5. **Cost & time overruns** — the most famous cause of project failure (see Unit II).
6. **User adoption resistance** — technically correct systems fail if users reject them.
7. **Stakeholder communication** — business users, developers and management speak different "languages".
8. **Technology obsolescence** — rapid change makes long design cycles risky.

🏭 Real-life use case — A real example

A bank replacing its old core system hits **all** of these: regulations change mid-project (requirements), millions of new UPI users arrive (scalability), the old mainframe must still talk to the new app (integration), and branch staff refuse to learn the new screens (adoption).

⚠️ Careful

Your notes' mnemonic **C-S-I-S-C-U-T** covers only **7** of the **8** challenges — it leaves out "stakeholder communication". Write all eight in the answer.

🧠 Yaad rakho

"**Badlav, Badhna, Jodna, Bachana, Kharcha, Kabool, Takneek**" — requirements badlav, scalability badhna, integration jodna, security bachana, cost kharcha, user adoption kabool karna, technology takneek.

📖 GFG: Challenges in Information System Design

1.3 Elements of Information System Design

📘 Definition

The **elements of an IS** are its six building blocks: People, Hardware, Software, Data, Procedures/Policies and Network/Communication.

Element	What it contains
People	End users, managers, IT staff, stakeholders
Hardware	Servers, client devices, networking equipment
Software	System software (OS, DBMS) + application software
Data	The resource being processed — databases, files, knowledge bases
Procedures / Policies	Rules governing how the system is operated (SOPs)
Network / Communication	Enables data flow between components, especially in distributed/cloud systems

⭐ **Exam tip**

Most short answers list five elements (People, Hardware, Software, Data, Procedures). Add **Network** as the sixth for distributed/cloud systems to stand out.

📖 GFG: Components of Information System

1.4 Roles and Responsibilities in IS Design

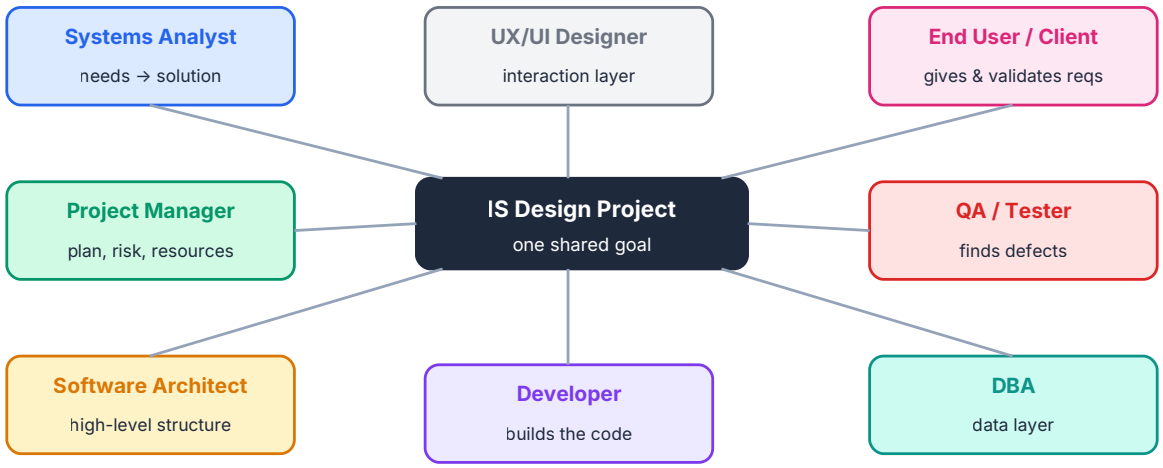


Fig. 5 — Eight roles around one shared project goal

Role	Responsibility
Systems Analyst	Bridges business needs and technical solutions; gathers requirements
Project Manager	Plans, schedules, allocates resources, manages risk
Software Architect	Designs the high-level structure of the system
Developer / Programmer	Implements the design as working code
Database Administrator (DBA)	Designs and manages the data layer
QA / Tester	Verifies the system meets requirements, finds defects
End User / Client	Gives requirements, validates the system, uses it after deployment
UX/UI Designer	Designs the interaction and interface layer

⭐ **Exam tip**

"Who does requirement elicitation?" → **Systems Analyst** (with input from end users/clients). "Who ensures quality?" → **QA/Tester** — but add that quality is a **shared responsibility** across the whole team. That nuance earns extra marks.

🏭 **Real-life use case — A small team in practice**

In a food-delivery app team: the **analyst** talks to restaurants and riders, the **architect** picks microservices, **developers** build them, the **DBA** designs the orders database, **QA** tests, the **UX designer** shapes the app flow and the **PM** keeps the launch date realistic.

📖 GFG: Role of a Systems Analyst in SDLC

1.5 Case Study Approach (IS Design)

📘 **Definition**

A **case-study question** gives a scenario (library system, hospital system…) and asks you to identify stakeholders, data entities, processes and challenges, then propose a design approach.

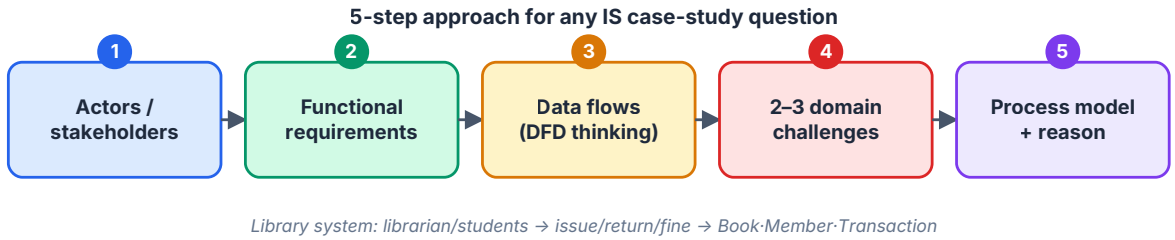


Fig. 6 — Five steps to answer any IS case-study question

1. **Identify actors / stakeholders.**
2. **List the major functional requirements.**
3. **Identify data flows** (Data-Flow-Diagram style thinking).
4. **Mention 2–3 challenges** specific to the domain.
5. **Suggest a process model** (Sec. 1.6) and justify it.

Library system	Answer
Stakeholders	Librarian, students, admin
Data entities	Book, Member, Transaction
Processes	Issue book, return book, fine calculation

The challenges and process-model choice for the library are left to you in the exam — e.g. "late-fee disputes, integration with student records" and "Agile, because the college keeps changing rules" (an illustration, not from the notes).

🧠 **Yaad rakho**

Case study = **"Kaun, Kya, Kaise, Kya-dikkat, Kaunsa-model"** — actors, requirements, data flow, challenges, model.

❓ Likely exam questions

- Design an information system for a library / hospital — identify stakeholders, entities and processes.
- Describe the case-study approach for IS design.
- Which process model would you suggest for a college ERP and why?

1.6 Software Processes & IS Design Models

📘 Definition

A **software process (process model)** is a structured set of activities used to develop software. **High-yield topic** — expect a direct comparison question.

(a) Traditional model — Waterfall

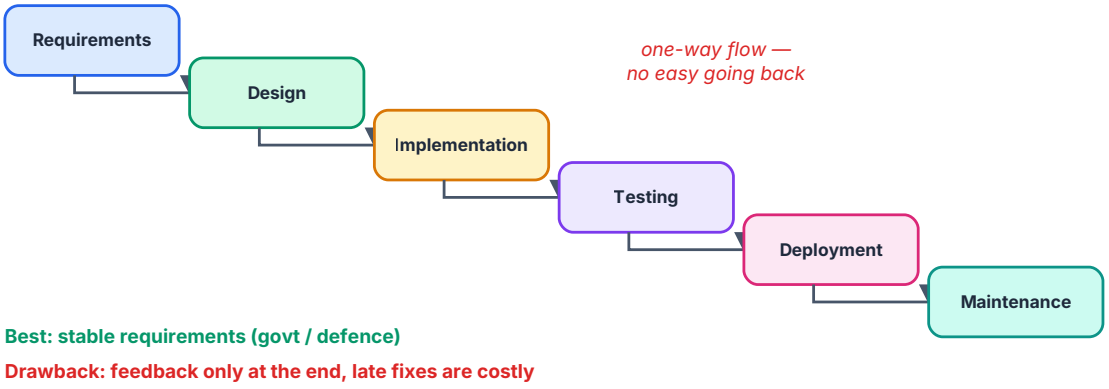


Fig. 7 — Waterfall — each phase completes before the next begins

- Sequential phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance.
- Each phase must finish before the next begins.
- **Best for:** well-understood, stable requirements (government / defence systems).
- **Drawback:** no feedback until late; errors found late are costly to fix.

🏭 Real-life use case

A railway signalling upgrade has fixed, regulated requirements that will not change for years — a good fit for Waterfall's heavy documentation and strict sign-offs.

📖 GFG: Waterfall Model

(b) Component-Based Development (CBD)

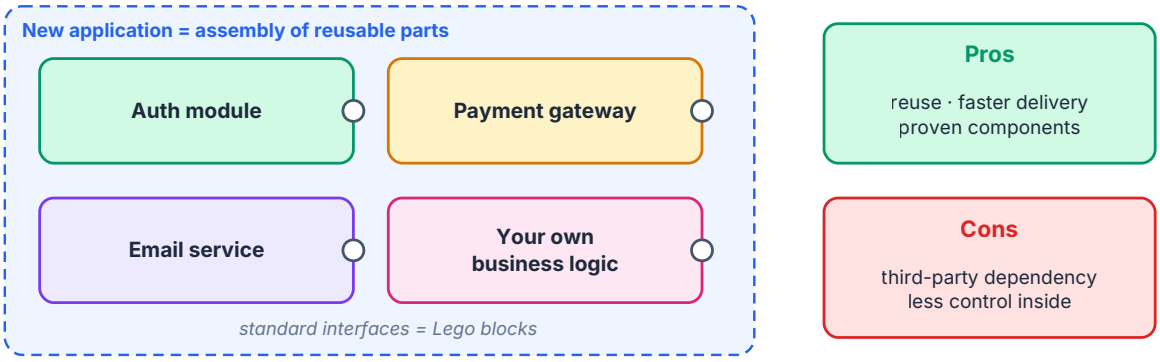


Fig. 8 — CBD — a new application assembled from reusable components with standard interfaces

- The system is built by **assembling pre-existing reusable components** rather than from scratch — like Lego blocks.
- Emphasises **reuse**, reduced development time and standardised interfaces.
- **Best for:** systems where proven components exist (payment gateway, auth module).
- **Drawback:** dependence on third-party components, integration challenges, less control inside a component.

🏭 Real-life use case

A start-up builds an online shop in weeks by plugging in a payment gateway, an email service and a login module, and writing only its own cart logic.

📖 GFG: Component-Based Software Engineering

(c) Agile Process

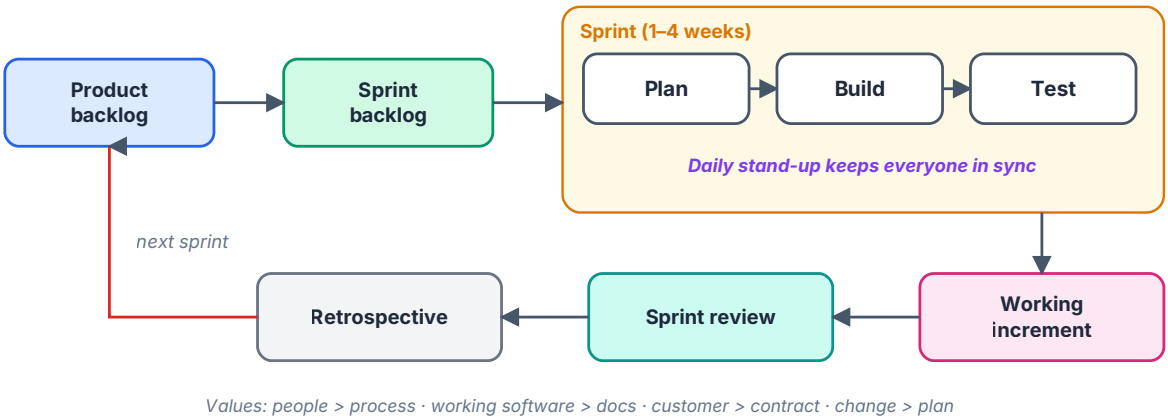


Fig. 9 — Agile — short sprints that each end with a working increment

- Iterative, incremental development in short cycles called **sprints** (typically 1–4 weeks).
- Agile Manifesto values: **individuals & interactions** over processes & tools; **working software** over documentation; **customer collaboration** over contract negotiation; **responding to change** over following a plan.
- Frameworks: **Scrum, Kanban, XP (Extreme Programming)**.
- **Best for:** evolving or unclear requirements and fast delivery.

- **Drawback:** needs a highly engaged customer; final cost/scope is hard to predict; documentation can be sparse.

Real-life use case

A fintech start-up releases a new feature of its payments app every two weeks and changes direction based on user feedback — the classic Agile situation.

📖 GFG: Agile Software Process Model

(d) Rational Unified Process (RUP)

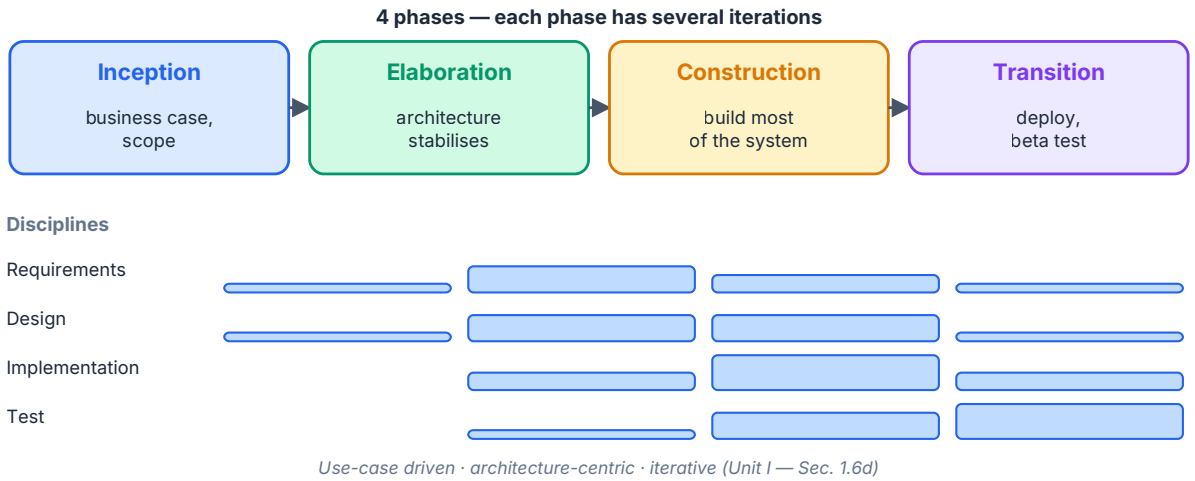


Fig. 10 — RUP — four phases, each containing several iterations

- An **iterative** model organised around **4 phases**: Inception → Elaboration → Construction → Transition.
- Structured around **UML**; **use-case driven** and **architecture-centric**.
- Combines the discipline of Waterfall with the iterative flexibility of Agile.
- **Best for:** large, complex enterprise systems needing formal documentation **and** iterative refinement.

Phase	Focus (from the notes)
Inception	Business case, scope
Elaboration	Architecture stabilises
Construction	Build most of the system
Transition	Deploy, beta testing

The "disciplines" bars in Fig. 10 are a simplified illustration of how effort shifts between phases — they are not exact data from the notes.

📖 GFG: Rational Unified Process (RUP) Model

Comparative table — very likely exam question

Aspect	Waterfall	CBD	Agile	RUP
Flexibility to change	Very low	Medium	Very high	Medium-High
Documentation	Heavy	Medium	Minimal	Heavy
Customer involvement	Only start/end	Medium	Continuous	Continuous (per iteration)
Risk handling	Late detection	Depends on component quality	Early, continuous	Early, risk-driven
Reusability focus	Low	Very high	Low-Medium	Medium
Best suited for	Stable, well-defined requirements	Available reusable components	Dynamic, evolving requirements	Large, complex, architecture-critical
Delivery style	Single final delivery	Assembled from components	Incremental (sprint-wise)	Iterative (phase-wise)

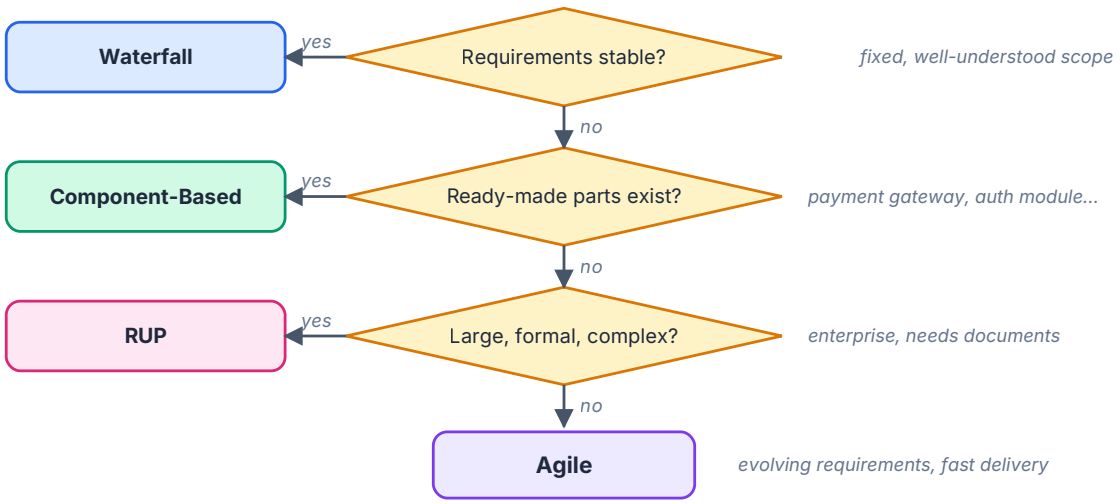


Fig. 11 — Quick decision guide — which process model fits your project?

> ⭐ **Exam tip — Answering "comparative study" questions**
>
> For **each** model write: (1) one-line definition, (2) diagram/flow, (3) advantages, (4) disadvantages, (5) best-use scenario — then finish with the summary table above. The decision flow in Fig. 11 is a revision aid built from the "best for" lines in your notes.

> 🧠 **Yaad rakho**
>
> Waterfall = **seedhi sadak**. CBD = **Lego**. Agile = **chhote chhote sprint**. RUP = **4 phase + iterations** (I-E-C-T).

> ⚠️ **Careful**
>
> Do not write that Agile has "no documentation" — the notes say **minimal / can be sparse**. And Waterfall is not "bad": it is the right choice when requirements are stable.

❓ Likely exam questions

- Compare Waterfall, CBD, Agile and RUP in a table.
- Explain Component-Based Development with advantages and drawbacks.
- State the four values of the Agile Manifesto.
- Explain the four phases of RUP.
- Distinguish Information System and Information Technology.
- Discuss the challenges faced in designing an information system.
- List the roles in IS design and the responsibility of each.

Unit I references: Pressman, *Software Engineering: A Practitioner's Approach* (process models) · Sommerville, *Software Engineering*, Ch. 2 · Agile Manifesto · IBM RUP overview · GFG: Software process models

UNIT II · CO3

Project Management and Planning

Iron triangle | Success & failure | Life cycle | Team structures | Metrics | COCOMO | Brooks's Law | Gantt / PERT | SCM | Risk

2.1 Project Management Essentials

📘 **Definition**

Software Project Management = planning, organising, staffing, monitoring and controlling a software project to meet its objectives within **scope, time, cost and quality** constraints.

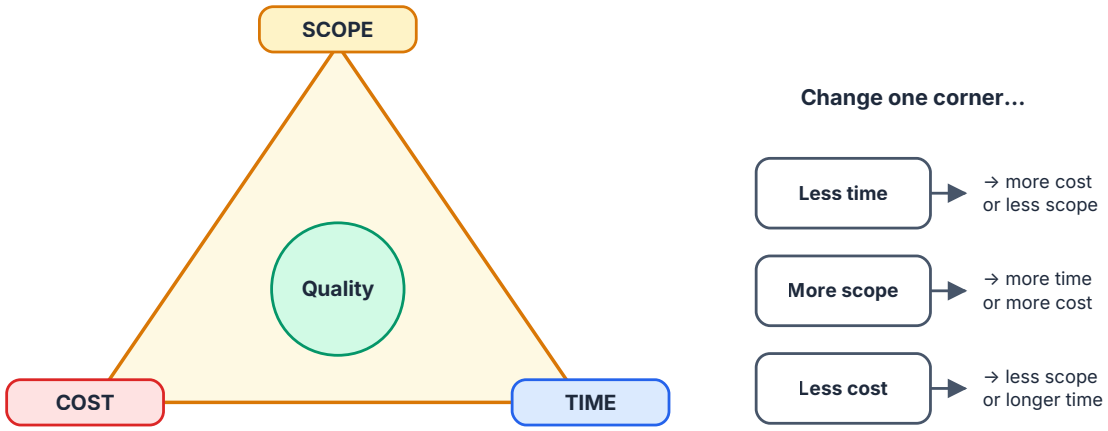


Fig. 12 — The Iron Triangle — scope, cost and time, with quality in the centre

The **Iron Triangle** (Triple Constraint): change one corner and another must give. Quality sits in the centre and is affected by all three.

🏭 **Real-life use case — A real squeeze**

A client suddenly asks to launch a month earlier. The manager has only three levers: add money (more developers — but see Brooks's Law), cut features (scope), or accept lower quality. That is the triangle in action.

⭐ **Exam tip**

Draw the triangle with **Scope, Cost, Time** on the corners and **Quality** in the middle — your notes call this a classic exam diagram. Add one example of a trade-off.

🧠 **Yaad rakho**

"**SCT + Q**": Scope, Cost, Time ke beech mein Quality. Ek kone ko kheencho, doosra khud khinch jaata hai.

📖 GFG: Iron Triangle / Triple Constraint

2.2 Project Success and Failure

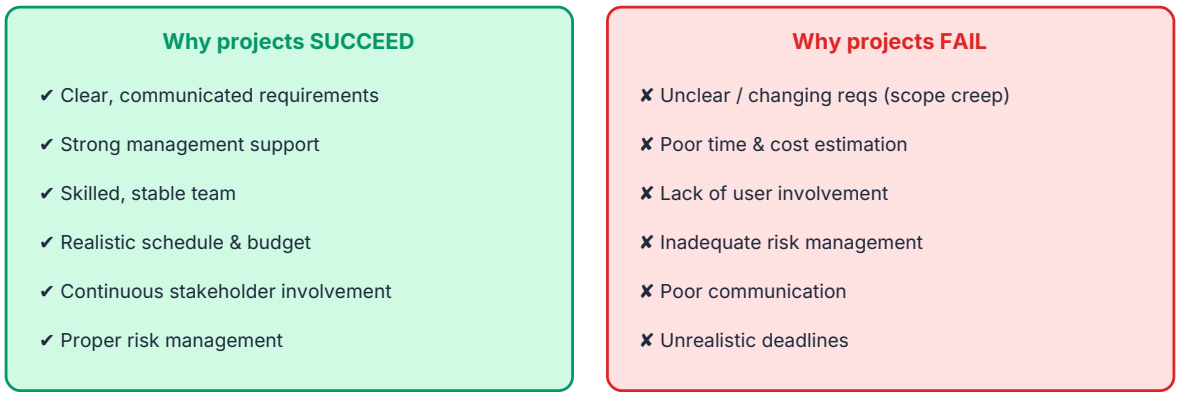


Fig. 13 — Reasons projects succeed and reasons they fail — mirror images of each other

Reasons for success	Reasons for failure (very commonly asked)
Clear, well-communicated requirements	Unclear / constantly changing requirements ("scope creep")
Strong executive / management support	Poor estimation of time and cost
Skilled and stable team	Lack of user involvement
Realistic scheduling and budgeting	Inadequate risk management
Continuous stakeholder involvement	Poor communication among stakeholders
Proper risk management	Unrealistic deadlines imposed by management

⭐ **Exam tip**

Name the **CHAOS Report by the Standish Group** — the most-cited industry source on software project success/failure rates. It adds credibility to your answer.

💡 **Why it matters**

Almost every failure reason is the **opposite** of a success reason. If you remember one list, you can write the other by flipping it.

2.3 Project Life Cycle

📘 **Definition**

The **project life cycle** is the generic set of phases a project passes through, mapped onto whichever process model the team uses.

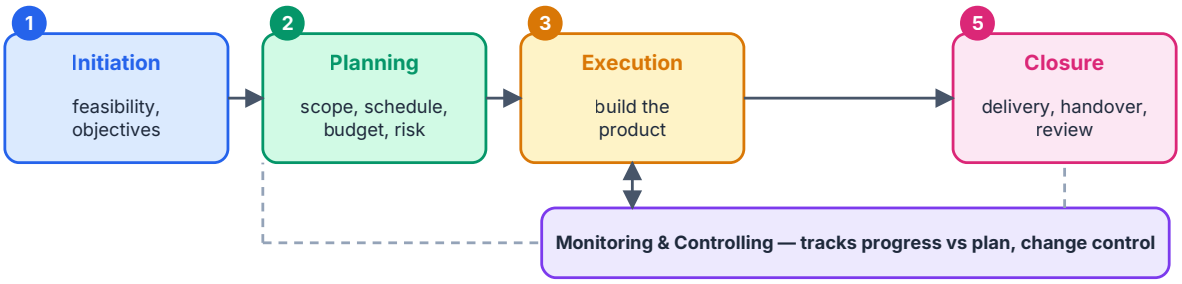


Fig. 14 — Initiation → Planning → Execution → Closure, watched throughout by Monitoring & Controlling

1. **Initiation** — feasibility study, define objectives.
2. **Planning** — scope, schedule, budget, resource plan, risk plan.
3. **Execution** — the actual development / build work.
4. **Monitoring & Controlling** — track progress vs plan, change control.
5. **Closure** — delivery, documentation handover, post-implementation review.

> ⚠️ **Careful**
>
> Monitoring & Controlling is step 4 in the list but it **runs alongside** planning and execution — that is why Fig. shows it as a bar under the phases, not as a separate box in the chain.

2.4 Project Team Structure and Organisation

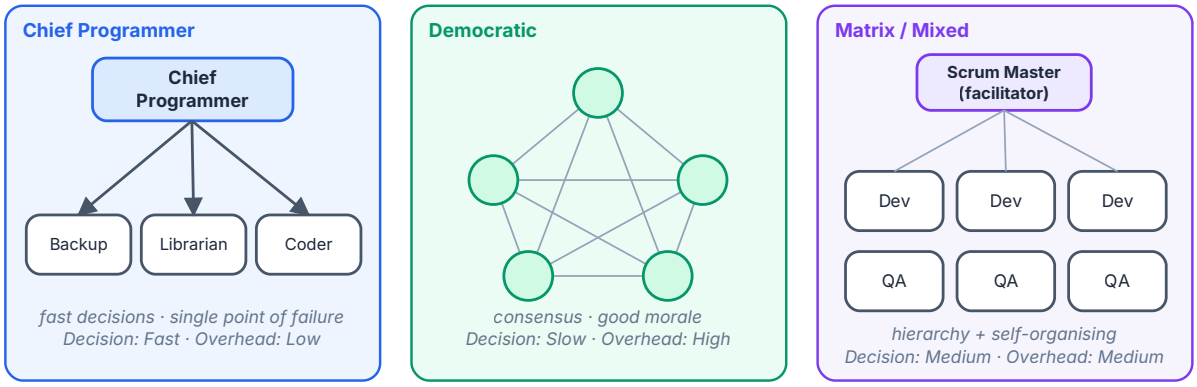


Fig. 15 — Three common team structures: chief programmer, democratic and matrix

Structure	Idea	Decision speed	Comm. overhead	Best for
Chief Programmer	One highly skilled lead + supporting specialists; single point of failure	Fast	Low	Small, well-defined projects
Democratic	No fixed leader; group consensus; good morale	Slow	High	Research / complex problems needing many views
Matrix / Mixed	Hierarchy + collaboration; Scrum Master facilitates, team self-organises	Medium	Medium	Agile / modern enterprise projects

🏭 **Real-life use case**

A 3-person start-up MVP suits a **chief programmer** setup; a research lab designing a new algorithm suits a **democratic** team; a product company running Scrum is a **matrix/mixed** team.

🧠 **Yaad rakho**

Chief = **ek boss, fast**. Democratic = **sab ki raay, slow**. Matrix = **beech ka raasta**.

2.5 Project Planning: Metrics and Measurements

📘 **Definition**

Metrics are quantifiable measures used to track project attributes.

Metric type	Examples
Size	Lines of Code (LOC), Function Points (FP)
Productivity	LOC / person-month, FP / person-month
Quality	Defect density (defects per KLOC)
Effort / cost	Person-months, cost per feature

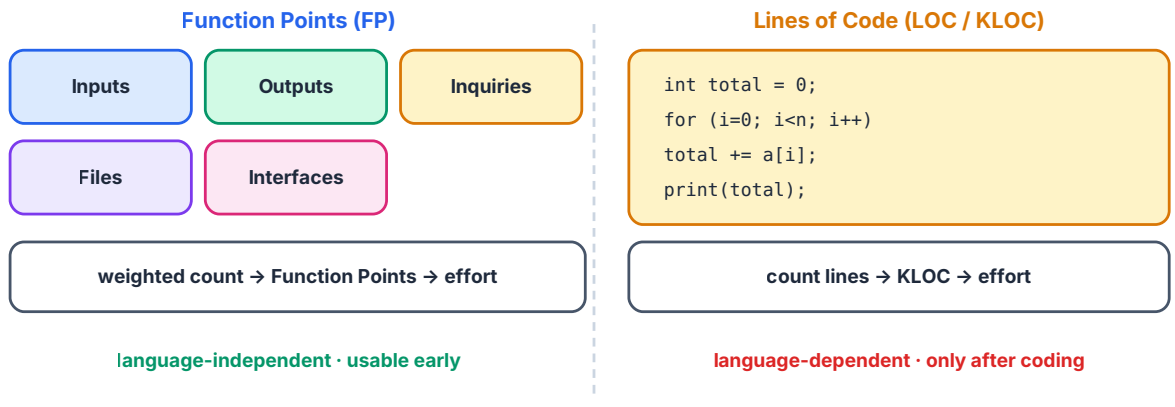


Fig. 16 — Function Points measure delivered functionality; LOC measures raw code volume

Function Points	Lines of Code
Language-independent	Language-dependent
Measures functionality delivered to the user	Measures raw code volume
Better for early estimation (before coding)	Measurable only after / during coding
More complex to calculate	Simple to count

> ⭐ **Exam tip**
>
> FP vs LOC is an exam favourite. The four contrasts to memorise: **language-independent vs dependent · functionality vs volume · early vs late · complex vs simple**.

2.6 Project Estimation — Effort Estimation Techniques

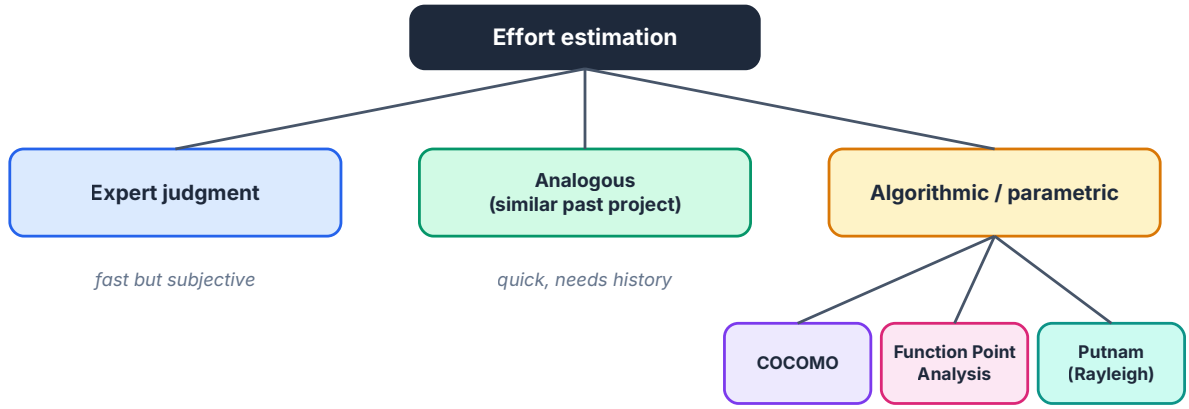


Fig. 17 — Three families of effort-estimation techniques

1. **Expert judgment** — based on the experience of senior developers/managers. Fast but subjective.
2. **Analogous estimation** — compare with a similar past project.
3. **Algorithmic / parametric models** —
   - **COCOMO** (Constructive Cost Model, Barry Boehm): effort from KLOC.
   - **Function Point Analysis (FPA)**: functional size (inputs, outputs, inquiries, files, interfaces) → Function Points → effort.
   - **Putnam model**: uses a Rayleigh curve to relate effort, schedule and staffing over time.

COCOMO model

> 📘 **Definition**
>
> **COCOMO** estimates effort from size: **Effort = a × (KLOC)^{b}** person-months, where **a** and **b** depend on the project type — Organic, Semi-detached or Embedded.

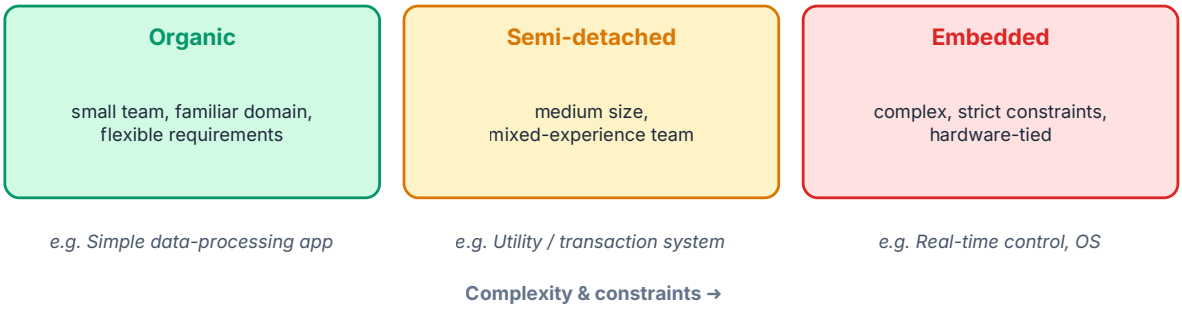
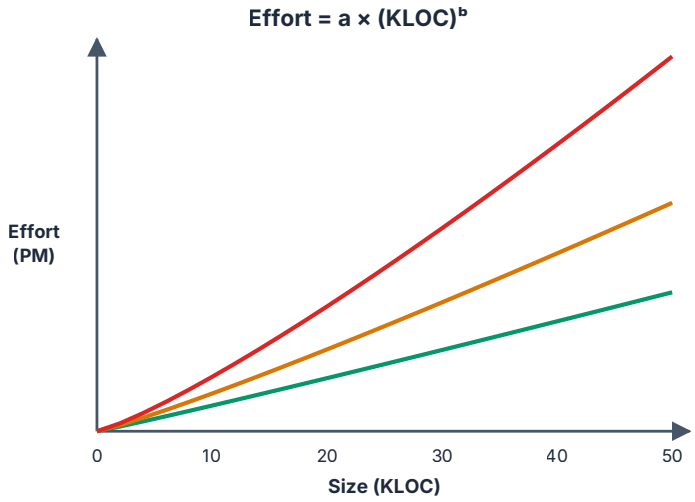


Fig. 18 — The three COCOMO project types

Type	Description	Example
Organic	Small team, familiar domain, flexible requirements	Simple data-processing app
Semi-detached	Medium size, mixed-experience team	Utility / transaction processing system
Embedded	Complex, strict constraints, hardware-tied	Real-time control systems, OS



Mode	a	b	PM @32
Organic	2.4	1.05	91
Semi-detached	3.0	1.12	146
Embedded	3.6	1.2	230

worked example: 32 KLOC basic COCOMO constants

same size → embedded needs ~2.5× organic effort

Fig. 19 — Effort grows faster than size, and faster still as project complexity rises

⚠️ Careful — Constants are not in your notes

Your notes give only the formula and say a and b depend on project type. The values used in Fig. above — Organic a=2.4, b=1.05; Semi-detached 3.0, 1.12; Embedded 3.6, 1.20 — are the **standard Basic COCOMO constants from textbooks** (Boehm). Use them only if your course book uses the same, and quote the formula first.

🏭 Real-life use case — Worked example (illustration, not from the notes)

For a 32 KLOC project, Basic COCOMO gives about **91** person-months (organic), **146** (semi-detached) and **230** (embedded). Same size, but the embedded system needs roughly 2.5× the effort — complexity and constraints are what cost money.

🧠 Yaad rakho

COCOMO = "**a ghatao b ki power**" — Effort = a × KLOC^b. Organic sasta, Embedded sabse mehnga.

📖 GFG: COCOMO Model

2.7 Staffing and Personnel Planning

- Match team members' skills to task requirements.
- The **personnel plan** includes: hiring plan, training needs, role assignment and a **resource histogram / staffing curve**.

📘 Definition — Brooks's Law

"Adding manpower to a late software project makes it later." — new people need training and add communication overhead that outweighs their extra capacity.

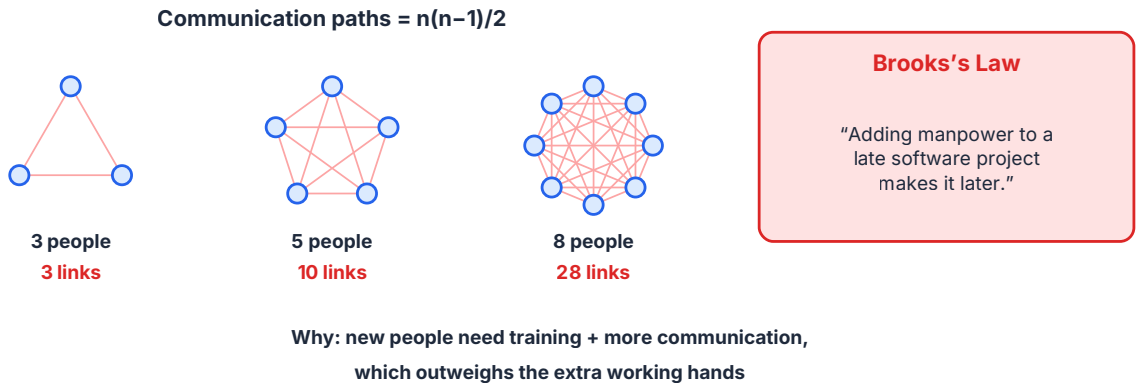


Fig. 20 — Communication paths grow as n(n−1)/2 — more people, many more links

💡 Why it matters

With n people there are n(n−1)/2 possible communication paths: 3 people → 3, 5 → 10, 8 → 28. Add people late and veterans spend time training newcomers instead of working. (The formula is standard textbook material — the notes give only the law and its reason.)

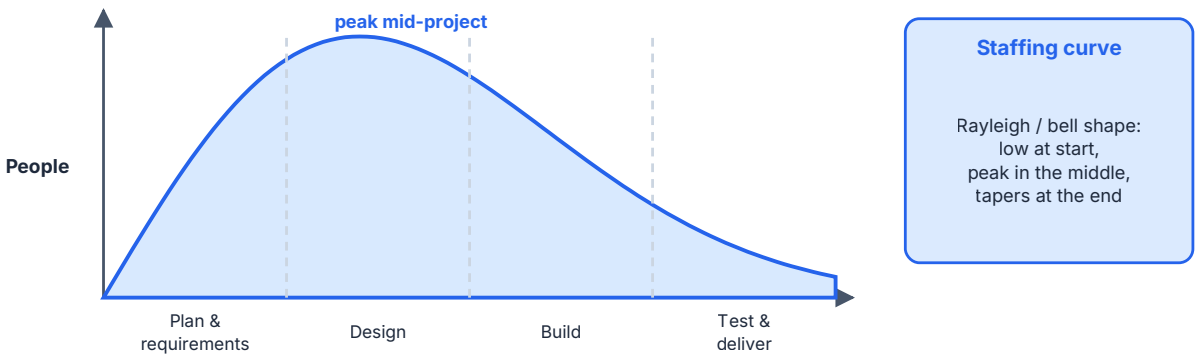


Fig. 21 — Staffing curve — low at the start, peak in the middle, taper at the end

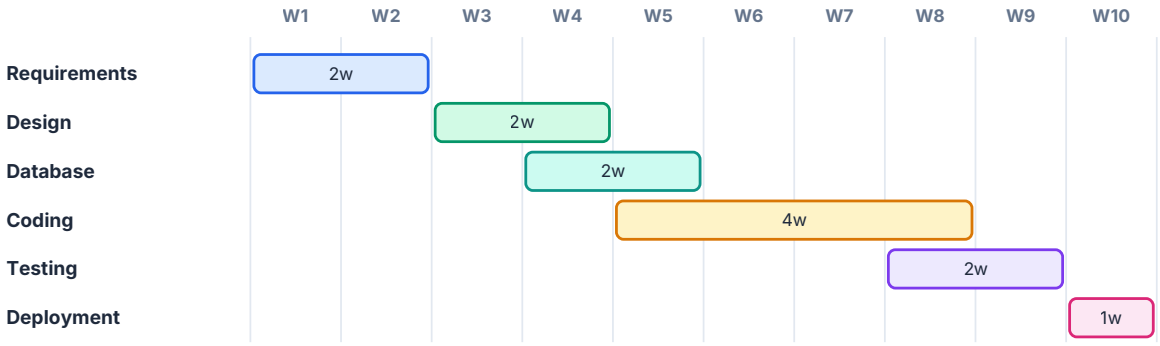
The staffing curve is usually bell/Rayleigh-shaped: few people at the start, a peak mid-project, tapering towards the end.

> ⭐ **Exam tip**
>
> Brooks's Law is "very frequently quoted in exams". Write the quote word-for-word, give the **reason** (training + communication overhead) and the 3-5-8 people example.

> 🧠 **Yaad rakho**
>
> Late project mein aur log daaloge to **aur late** — nayon ko sikhana padta hai aur baat-cheet ke raaste badh jaate hain.

2.8 Project Scheduling

Tool	What it is
Gantt chart	Bar chart of tasks against a timeline — shows overlaps and duration but not dependencies clearly
PERT chart	Program Evaluation Review Technique — network diagram of task dependencies and the **critical path**: the longest dependent sequence, which fixes the minimum project duration
CPM	Critical Path Method — identifies the critical path; any delay on it delays the whole project

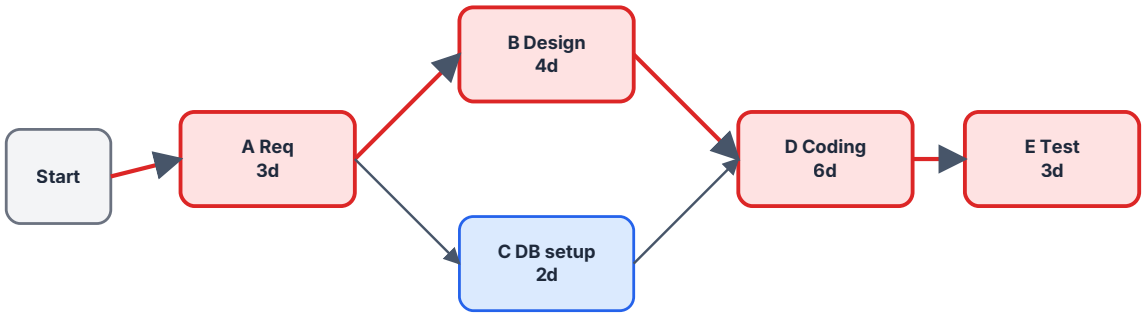


Bars show timeline & overlaps — but NOT task dependencies

Database overlaps Design (weeks 4) — visible at a glance

Fig. 22 — Gantt chart — tasks as bars on a week timeline

Critical path = A → B → D → E = 3 + 4 + 6 + 3 = 16 days



Path A → C → D → E = 3 + 2 + 6 + 3 = 14 days → 2 days of slack

Any delay on a red (critical) task delays the whole project

Fig. 23 — PERT / CPM network — the red path is the critical path

> ⚠️ **Careful — Numbers are an illustration**
>
> The tasks, durations and 16-day critical path in these two figures are a made-up example to show the idea — they are not from your notes.

Gantt chart	PERT / CPM
Timeline / bar based	Network / node based
Easy to read, shows schedule	Shows task dependencies clearly
Does not highlight the critical path directly	Explicitly identifies the critical path
Good for simple / small projects	Good for complex projects with many dependencies

> 🧠 **Yaad rakho**
>
> Gantt = **kab** (timeline). PERT = **kiske baad kya** (dependency + critical path).

2.9 Software Configuration Management (SCM)

> 📘 **Definition**
>
> **SCM** is the discipline of controlling and tracking changes to software artefacts (code, documents, requirements) throughout the project life cycle.

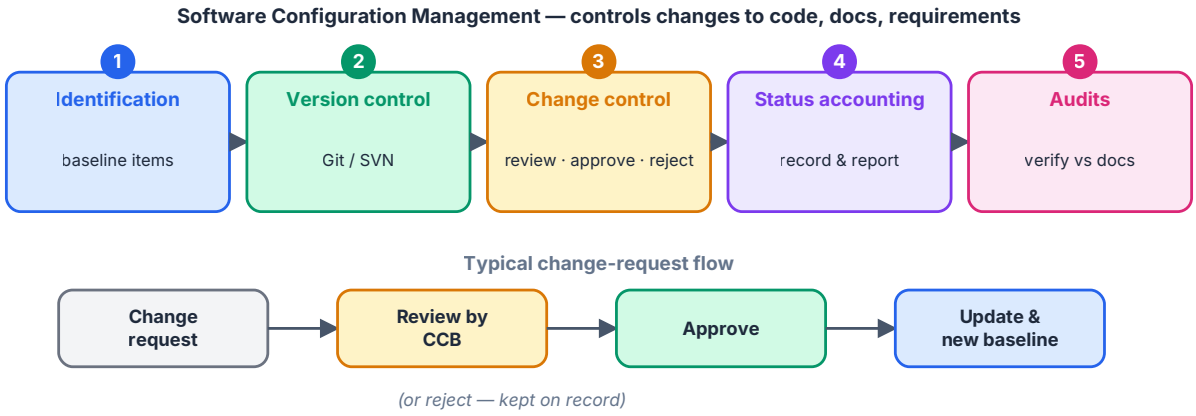


Fig. 24 — The five SCM activities and a typical change-request flow

1. **Configuration identification** — identify the items to be controlled (a "baseline").
2. **Version control** — track changes over time (Git, SVN).
3. **Change control** — formal process to review, approve or reject change requests.
4. **Configuration status accounting** — record and report the status of configuration items.
5. **Configuration audits** — verify the delivered product matches its documented configuration.

> 🏭 **Real-life use case**
>
> A team using Git keeps the code under version control; a request to change the tax rule goes through a review board (CCB — a standard SCM term, not in your notes) before a new baseline is cut and released.

🧠 **Yaad rakho**

SCM = **I-V-C-S-A**: Identification, Version, Change, Status, Audit.

2.10 Risk Management

📘 **Definition**

Risk management is the process of identifying, analysing, prioritising, mitigating and monitoring things that could go wrong in a project.

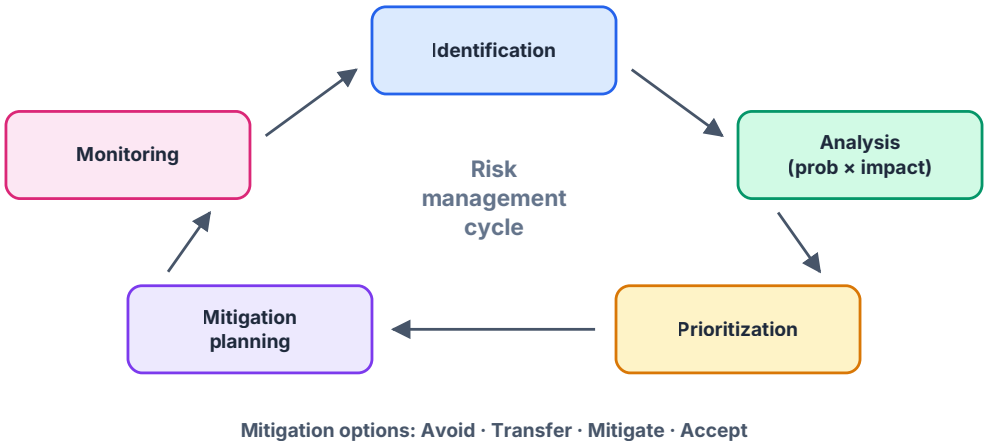


Fig. 25 — The risk-management cycle

1. **Identification** — brainstorm potential risks (technical, schedule, cost, resource, external).
2. **Analysis** — assess probability and impact (often on a probability–impact matrix).
3. **Prioritisation** — rank risks by severity = probability × impact.
4. **Mitigation** — plan to reduce probability or impact: **avoid, transfer, mitigate, accept**.
5. **Monitoring** — track identified risks throughout the project.

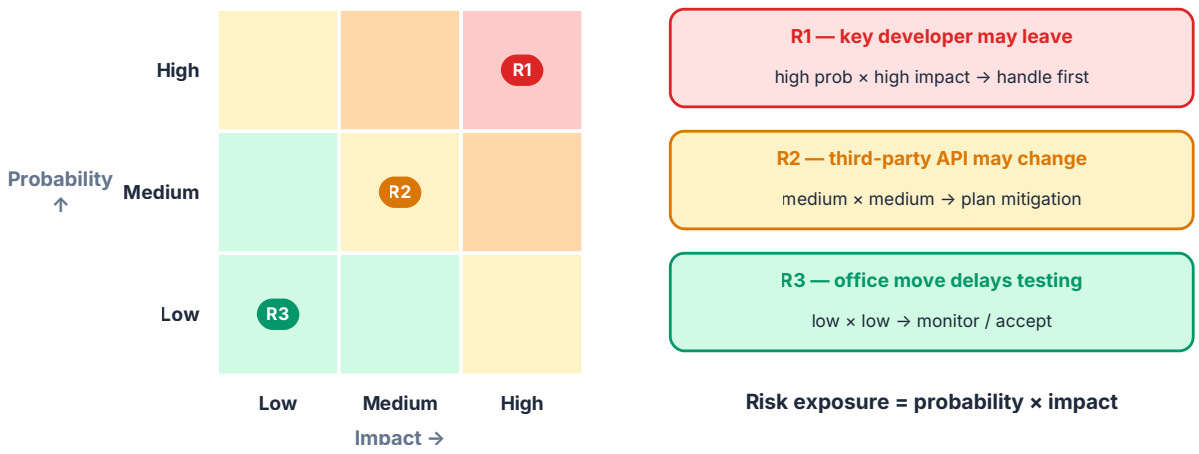


Fig. 26 — Probability–impact matrix — handle the top-right risks first



Brainstorm risks under all three headings during Risk Identification

Fig. 27 — Three types of risk

Type	Meaning
Project risk	Schedule / resources
Technical risk	Design / implementation issues
Business risk	Market / organisational

⭐ **Exam tip**

Write the cycle in order **Identify → Analyse → Prioritise → Mitigate → Monitor**, draw the loop, and mention **exposure = probability × impact** and the four responses (avoid / transfer / mitigate / accept).

🧠 **Yaad rakho**

Risk = **kitni baar × kitna nuksaan**. Pehle bade risk ko pakdo.

❓ **Likely exam questions**

- Explain the Iron Triangle with a diagram.
- Why do software projects fail? Give reasons and cite the CHAOS Report.
- Explain the project life cycle phases.
- Compare Chief Programmer, Democratic and Matrix teams.
- Function Points vs Lines of Code.
- Explain COCOMO and its three project types.
- State and explain Brooks's Law.
- Differentiate Gantt chart and PERT/CPM. What is the critical path?
- Explain the SCM activities.
- Explain the risk-management process with a diagram.

Unit II references: Pressman — project management, estimation (COCOMO), risk · Standish Group CHAOS Report · GFG: Software project management · PERT vs Gantt

UNIT III · CO4

Requirements Engineering & Software Design

RE cycle | Elicitation | Structured vs OO | Prototyping | SRS | FR / NFR | Validation | Coupling & cohesion | Design process | Refactoring

3.1 Requirements Engineering (RE) — Overview

📘 **Definition**

Requirements Engineering is the process of **defining, documenting and maintaining** requirements.



Fig. 28 — RE cycle — elicitation, analysis, specification, validation, wrapped by requirements management

Four core sub-activities (draw as a cycle in exams): **Elicitation → Analysis → Specification → Validation**, with **Requirements Management** wrapped around all of them.

🧠 **Yaad rakho**

E-A-S-V + Management: "Elicit karo, Analyse karo, Spec likho, Validate karo — aur sab par Management ka chhata."

3.2 Requirements Elicitation

📘 **Definition**

Elicitation is the process of gathering requirements from stakeholders.

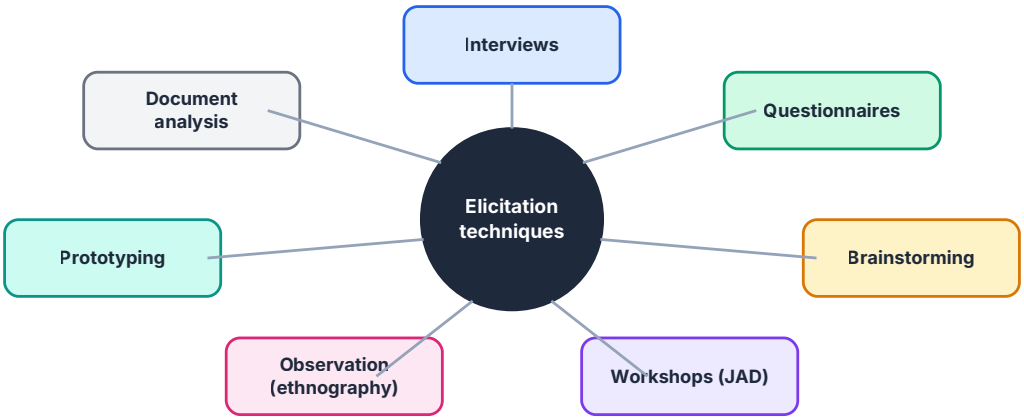


Fig. 29 — Seven elicitation techniques

Technique	Idea
Interviews	Talk to stakeholders one-to-one
Questionnaires / surveys	Collect answers from many people
Brainstorming	Generate ideas as a group
Workshops (JAD)	Joint Application Development sessions
Observation (ethnography)	Watch users in their natural work environment
Prototyping	Build a mock version to get feedback
Document analysis	Study existing system documentation

⚠️ Careful — Challenges in elicitation

Stakeholders don't always know what they want · requirements conflict between stakeholders · **tacit knowledge** that users can't put into words.

🏭 Real-life use case

For a hospital system, the analyst **observes** nurses at the ward (they cannot describe all their shortcuts), **interviews** doctors and sends a **questionnaire** to the billing staff.

3.3 Requirements Analysis: Structured vs Object-Oriented

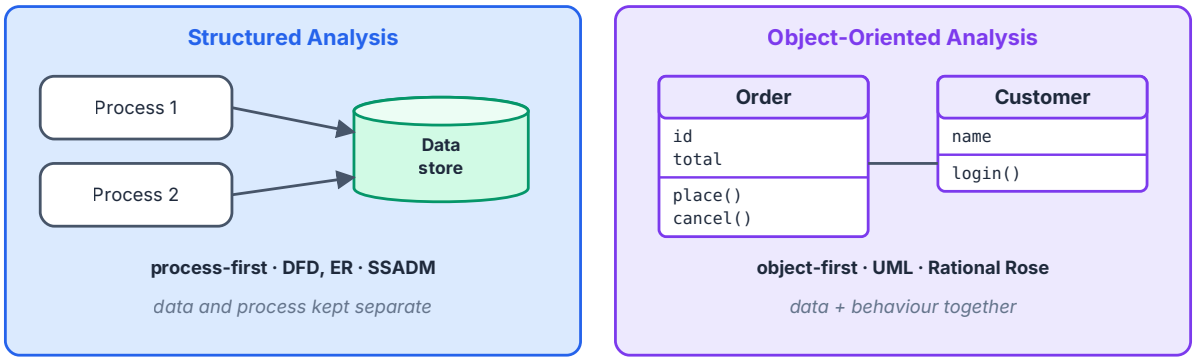


Fig. 30 — Structured analysis separates processes and data; OO analysis combines them in objects

Point	Structured Analysis	Object-Oriented Analysis
Starts with	Processes / functions (top-down decomposition)	Objects / entities and their interactions
Uses	DFD (Data Flow Diagrams), ER diagrams	UML diagrams (use case, class, sequence...)
Data & process	Separate	Combined in objects (encapsulation)
Good for	Well-defined, process-heavy systems (payroll)	Complex systems needing reuse/extensibility (modern web apps)
Example tool	SSADM	UML, Rational Rose

Prototyping analysis

Definition

Prototyping is building a working (often partial) model of the system early, to elicit and clarify requirements through user feedback.

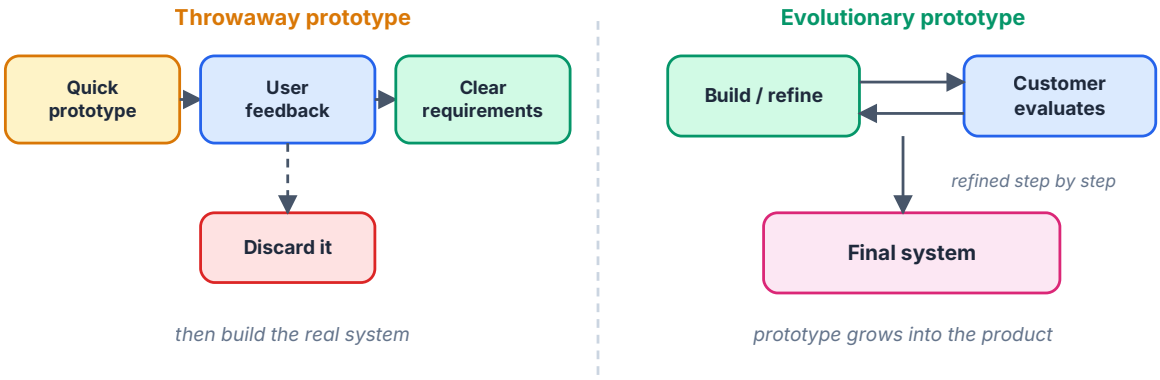


Fig. 31 — Throwaway prototype is discarded; evolutionary prototype grows into the final system

Type	What happens
Throwaway	Prototype is discarded after the requirements are clarified
Evolutionary	Prototype is refined step by step into the final system

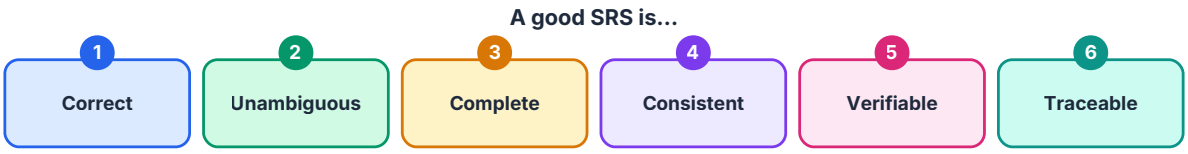
🧠 **Yaad rakho**

Throwaway = **seekho aur phenko**. Evolutionary = **seekho aur badhao**.

3.4 Requirements Specification (SRS)

📘 **Definition**

The **Software Requirements Specification (SRS)** is the formal, agreed-upon document describing **what** the system must do.



Traceable: each requirement links to its origin AND to design / test artifacts

Memory hook: C-U-C-C-V-T

Fig. 32 — Six qualities of a good SRS

Quality	Meaning
Correct	Accurately represents stakeholder needs
Unambiguous	Only one interpretation possible
Complete	Covers all functional and non-functional requirements
Consistent	No contradicting requirements
Verifiable	Testable against defined criteria
Traceable	Each requirement traced to its origin and to design/test artefacts

Functional vs non-functional requirements

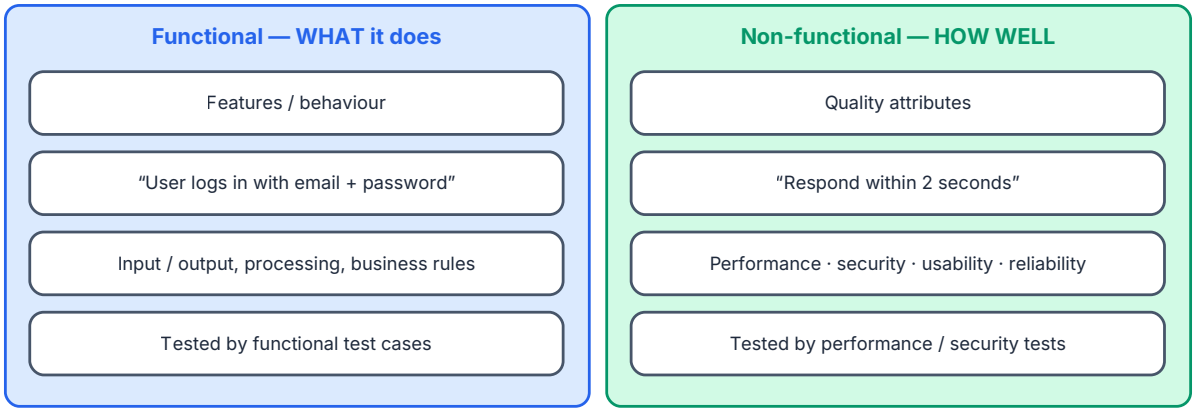


Fig. 33 — Functional = WHAT the system does; non-functional = HOW WELL it does it

Point	Functional	Non-functional
Meaning	WHAT the system should do (features/ behaviour)	HOW WELL it performs (quality attributes)
Example	"User can log in with email and password"	"System should respond within 2 seconds"
Testing	Functional test cases	Performance, security, usability testing
Categories	Input/output, processing logic, business rules	Performance, security, usability, reliability, scalability, maintainability

🧠 **Yaad rakho**

SRS ke 6 gun: **C-U-C-C-V-T** — Correct, Unambiguous, Complete, Consistent, Verifiable, Traceable.

3.5 Requirements Validation

📘 **Definition**

Requirements validation ensures the specified requirements really reflect what stakeholders need — **before** moving to design.

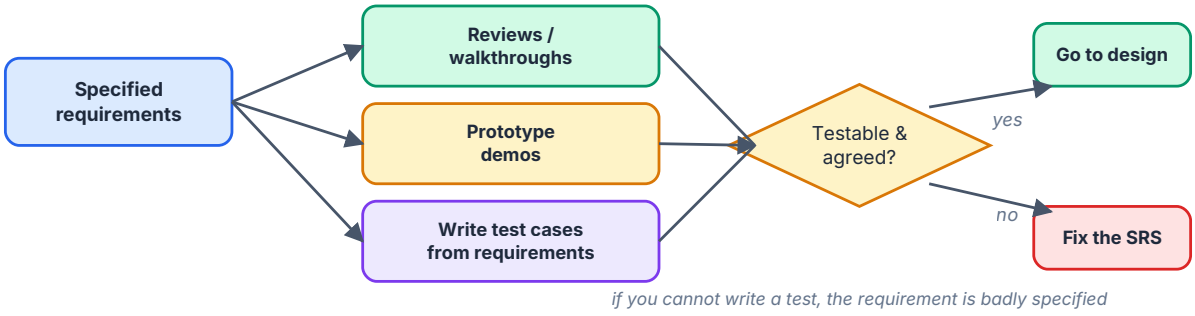


Fig. 34 — Validation techniques — reviews, prototype demos and test-case generation

💡 **Why it matters**

If you cannot write a test for a requirement, it is probably poorly specified. Writing test cases from requirements is itself a validation technique.

Techniques: requirement **reviews / walkthroughs**, **prototype demos**, and **test-case generation** from requirements.

3.6 Requirements Management and Tools

📘 **Definition**

Requirements management is handling changes to requirements over the project's life — including **traceability** and version control of requirement documents.

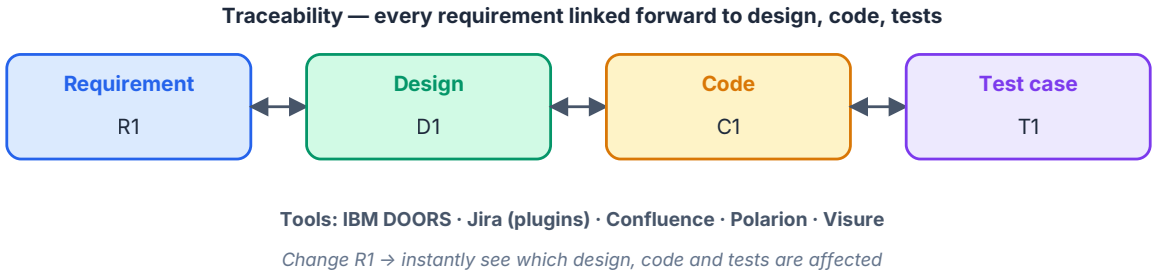


Fig. 35 — Traceability links each requirement forward to design, code and tests

Common tools: **IBM DOORS, Jira (with requirement plugins), Confluence, Polarion, Visure Requirements**.

🏭 **Real-life use case**

When a client changes "pay by card" to "pay by card or UPI", traceability instantly shows which design pages, code modules and test cases must be updated.

3.7 Software Design Fundamentals

📘 **Definition**

Design is the process of transforming requirements (the "what") into a blueprint for construction (the "how").

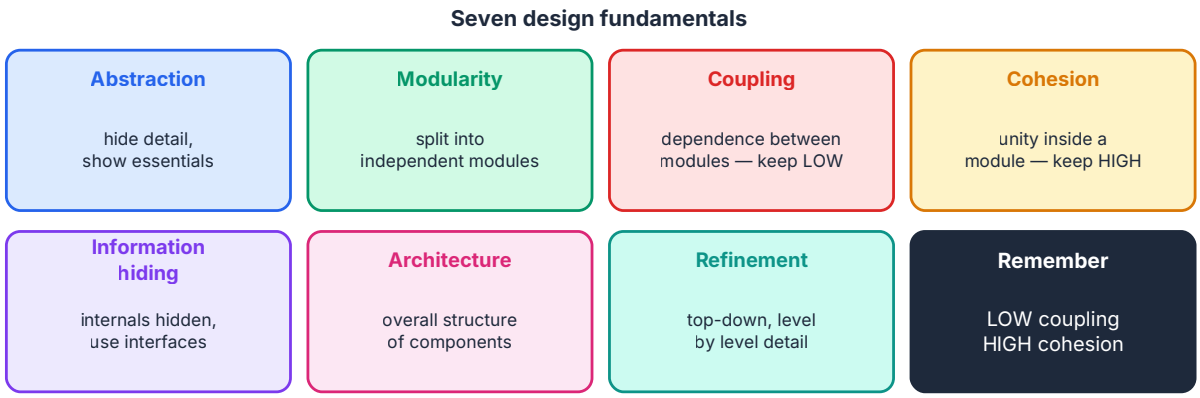


Fig. 36 — Seven design fundamentals

1. **Abstraction** — hide implementation detail, expose only essentials.
2. **Modularity** — divide the system into independent, manageable modules.
3. **Coupling** — degree of interdependence between modules. Goal: **LOW**.
4. **Cohesion** — how well elements *within* one module belong together. Goal: **HIGH**.
5. **Information hiding** — internal details hidden; modules interact only via defined interfaces.
6. **Architecture** — overall structure of components and their relationships.
7. **Refinement** — top-down strategy: elaborate the design in successive levels of detail.

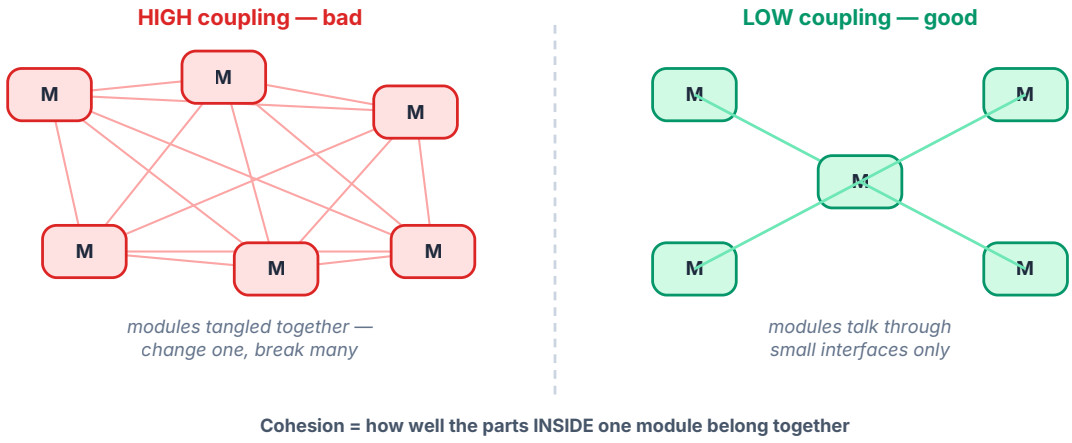


Fig. 37 — High coupling is a tangle; low coupling means modules talk through small interfaces

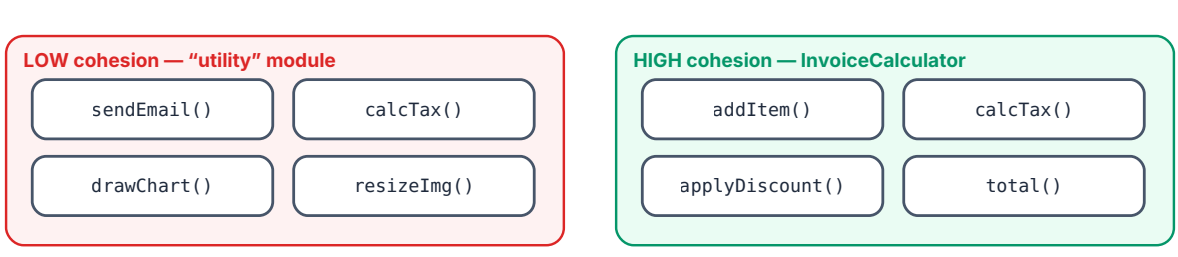


Fig. 38 — Low cohesion mixes unrelated jobs; high cohesion keeps one well-defined job

⭐ **Exam tip — Guaranteed question**

"What is coupling and cohesion? Which is preferred?" Answer: we want **LOW coupling and HIGH cohesion** — modules become easier to understand, test, maintain and reuse independently. The utility-module and InvoiceCalculator examples above are illustrations, not from the notes.

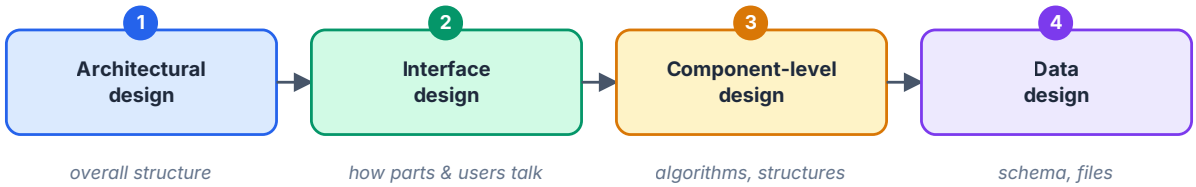
🧠 **Yaad rakho**

Coupling kam, Cohesion zyada. "Alag-alag module aapas mein kam jude, par har module ke andar sab kuch ek hi kaam ka ho."

⚠️ **Careful**

Students often swap the two. Coupling = **between** modules. Cohesion = **inside** one module.

3.8 Design Process



Architecture examples: layered · client-server · microservices · MVC

Requirements = WHAT → Design = HOW

Fig. 39 — Typical design flow — architecture, interface, component-level, data

Step	What it defines
Architectural design	Overall system structure (layered, client-server, microservices, MVC)
Interface design	How modules/components communicate with each other and with users (UI)
Component-level design	Detailed internal design of each component (algorithms, data structures)
Data design	How data is organised (database schema, file structures)

3.9 Modular Design and Component-Level Design

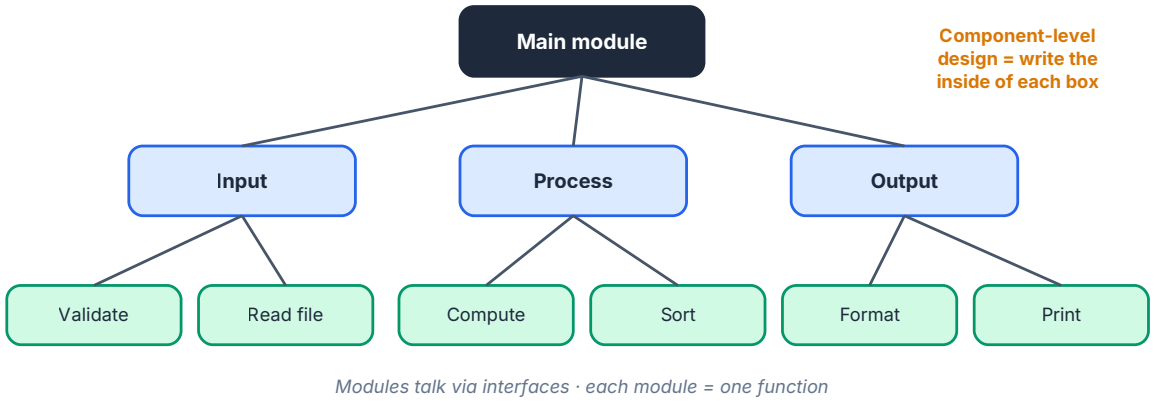


Fig. 40 — A main module split into sub-modules; component-level design fills in each box

- **Modular design** — break the system into discrete, manageable modules, each handling a specific function and communicating through well-defined interfaces.
- **Component-level design** — one level deeper: design the internal procedural detail of each module (pseudocode, flowcharts, algorithms) before coding.

3.10 Structured vs Object-Oriented Design

Point	Structured design	Object-oriented design
Driven by	Functions (top-down, step-wise refinement)	Objects (bottom-up, models real-world entities)
Chart	Structure charts for module hierarchy	UML class / object diagrams
Data & functions	Separate	Combined inside objects (encapsulation)
Reuse	Less reusable	Highly reusable (inheritance, polymorphism)
Example	Legacy COBOL / C systems	Modern Java / Python / C++ enterprise systems

3.11 Refactoring

📘 **Definition**

Refactoring is restructuring existing code **without changing its external behaviour** to improve internal quality — readability, maintainability, reduced technical debt.

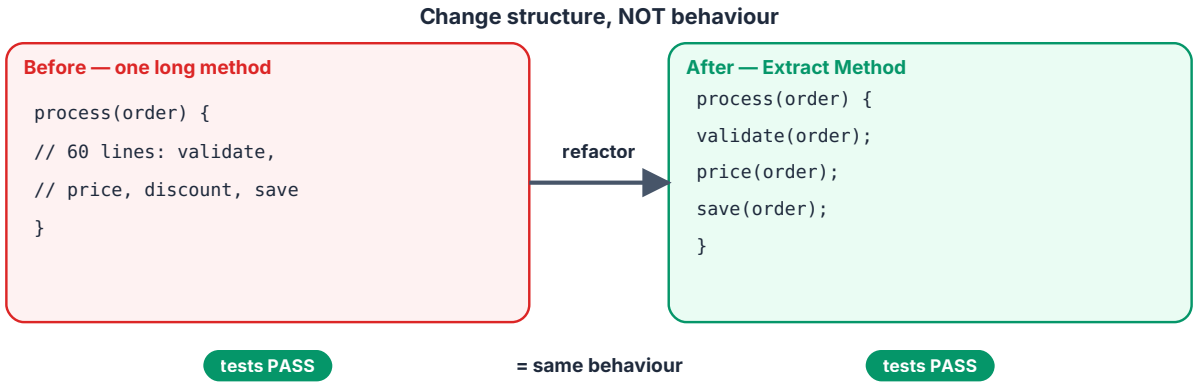


Fig. 41 — Refactoring changes structure but tests pass identically before and after

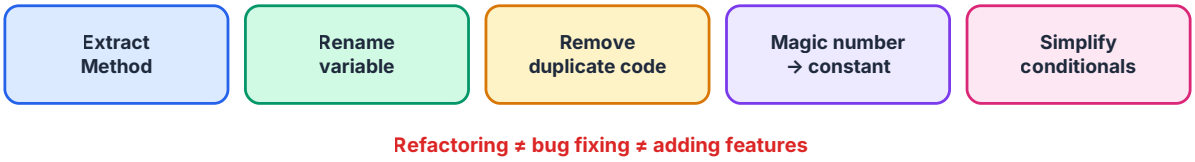


Fig. 42 — Common refactoring techniques

- **Extract Method** — break a long method into smaller ones
- **Rename variable/method** — for clarity
- **Remove duplicate code**
- **Replace magic numbers with named constants**
- **Simplify conditional expressions**

> ⚠️ **Careful**
>
> Refactoring is **not** bug fixing and **not** adding features. It changes structure, not behaviour — tests must pass identically before and after.

> 🧠 **Yaad rakho**
>
> Refactor = **ghar ki safai, naya kamra nahi**. Kaam wahi, tarika saaf.

> ❓ **Likely exam questions**
>
> - Explain the Requirements Engineering process with a diagram.
> - Discuss elicitation techniques and their challenges.
> - Compare Structured and Object-Oriented analysis.
> - Explain throwaway and evolutionary prototyping.
> - What are the qualities of a good SRS? Functional vs non-functional requirements.
> - Explain requirements validation and management.
> - What are coupling and cohesion? Which is preferred and why?
> - Explain the design process and the key design principles.
> - Compare structured and object-oriented design.
> - What is refactoring? Name techniques. How is it different from bug fixing?

Unit III references: Sommerville — Ch. 4 (Requirements Engineering), Ch. 7 (Design Engineering) · Martin Fowler, *Refactoring* · GFG: Requirements engineering · Coupling and cohesion

UNIT IV · CO5

Object-Oriented Analysis, Design & UML Modeling

UML | Use case | Class & object | Sequence | Collaboration | State-chart | Activity | Component | Deployment | Package | Case study

⭐ **Exam tip — Highest-weightage unit**

This is the most diagram-heavy unit — expect **at least one full UML diagram-drawing question**. Practise drawing every figure here 2–3 times by hand.

4.1 What is UML?

📘 **Definition**

UML (Unified Modeling Language) is a standardised, general-purpose visual modeling language used to **specify, visualise, construct and document** the artefacts of a software system, especially in object-oriented development.

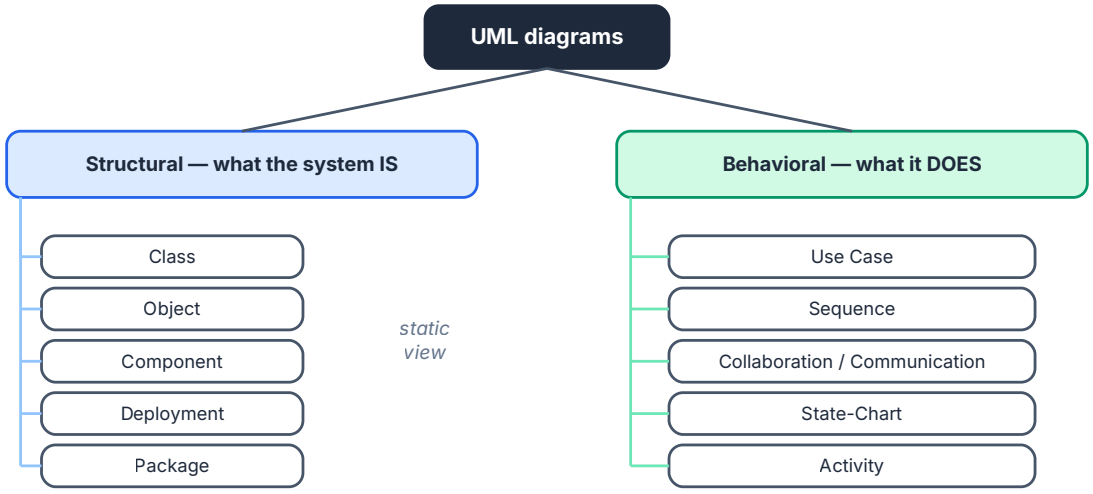


Fig. 43 — UML diagrams — structural (static) and behavioral (dynamic)

Category	Diagrams
Structural — what the system IS (static)	Class, Object, Component, Deployment, Package
Behavioral — what the system DOES (dynamic)	Use Case, Sequence, Collaboration/Communication, State-Chart, Activity

🧠 **Yaad rakho**

Structural = **"hai"** (kya hai). Behavioral = **"karta hai"** (kya karta hai). 5 + 5 diagrams.

4.2 Use Case Diagrams

📘 **Definition**

A **use case diagram** represents the functional requirements of a system from the **user's (actor's) perspective** — it shows **WHAT** the system does, not HOW.

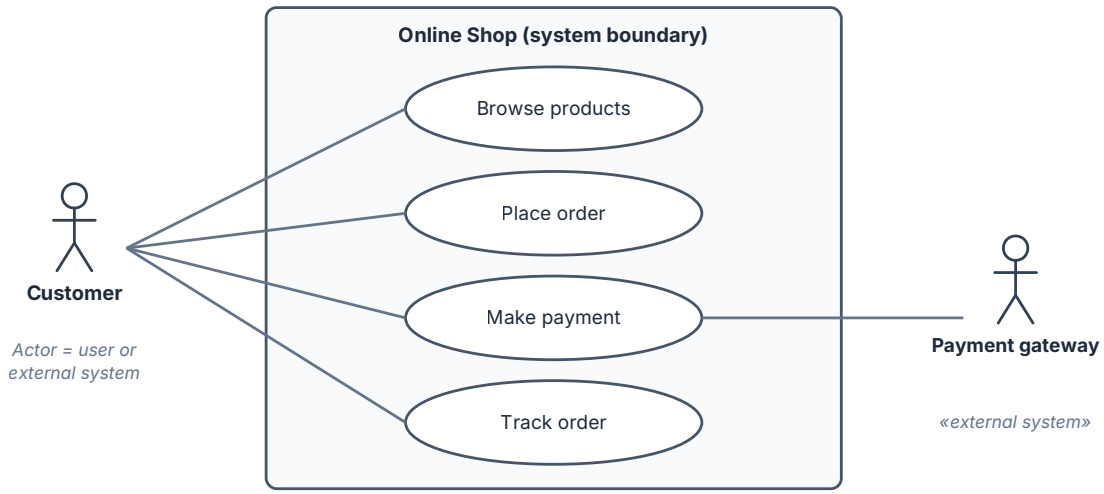


Fig. 44 — Use case diagram of an online shop — actors outside the boundary, use cases inside

Element	Notation
Actor	Stick figure — a user or external system
Use case	Oval — a specific functionality/goal (Login, Place Order)
System boundary	Rectangle enclosing all use cases — the scope of the system
Association	Line connecting actor to use case

Use case relationships — guaranteed exam question

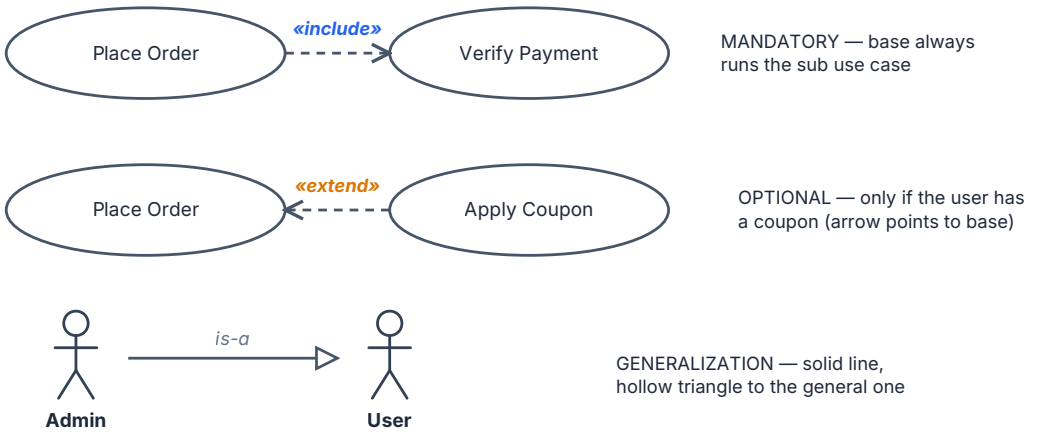


Fig. 45 — Include, extend and generalization at a glance

Relationship	Notation	Meaning
Include	«include» dashed arrow	**Mandatory** — base use case ALWAYS includes the sub use case (Place Order always includes Verify Payment)
Extend	«extend» dashed arrow	**Optional** — the sub use case MAY extend the base under a condition (Place Order may extend to Apply Discount Coupon only if the user has one)
Generalization	Solid line, hollow triangle	One use case/actor is a specialised version of another (Admin generalises from User)

Use case scenario — a textual, step-by-step narrative of how an actor achieves a goal. Document it with: **Use Case Name, Actor(s), Preconditions, Main Flow (steps), Alternate Flow, Postconditions**.

⚠️ **Careful**

The **extend** arrow points **from** the optional use case **to** the base use case. Most students draw it the wrong way round.

🧠 **Yaad rakho**

Include = Zaroori (always). **Extend = Extra** (optional). "I for Important, E for Extra."

🏭 **Real-life use case**

An ATM: "Withdraw cash" **includes** "Verify PIN" (every time) and may be **extended** by "Print receipt" (only if the customer asks).

4.3 Class and Object Diagrams

📘 **Definition**

A **class diagram** is the backbone of OO design: it shows classes, their attributes, methods and the relationships between them.

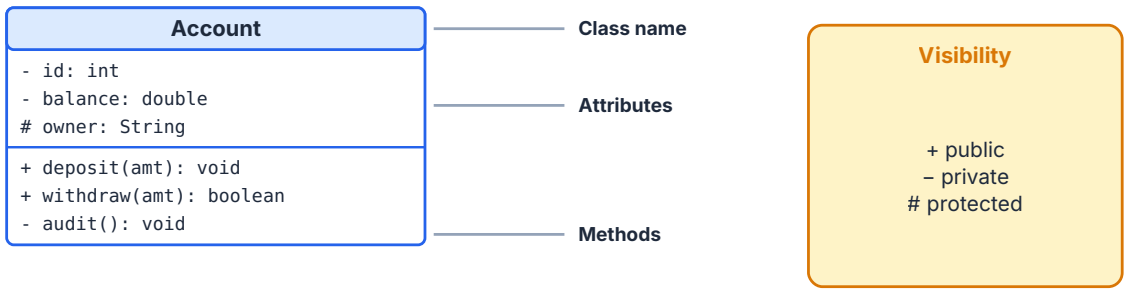


Fig. 46 — A class is a 3-part rectangle — name, attributes, methods

Visibility: **+** public, **–** private, **#** protected.

Relationships in class diagrams

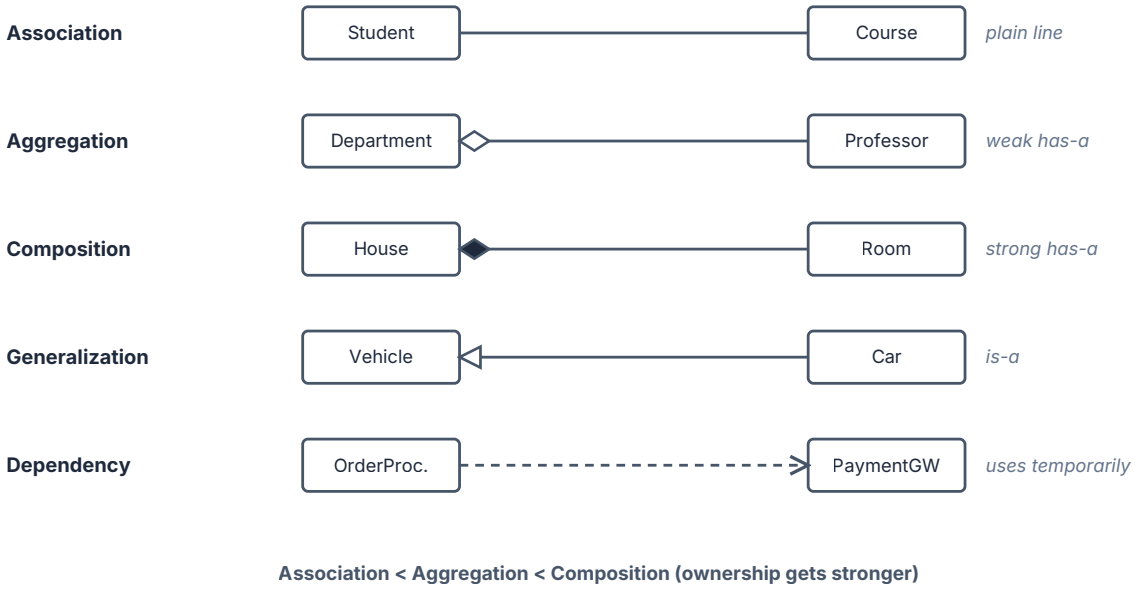


Fig. 47 — The five class relationships and their notation

Relationship	Notation	Meaning	Example
Association	Plain line	Two classes are related	Student — Course
Aggregation	Hollow diamond	"Has-a", **weak** ownership (part exists independently)	Department has Professors
Composition	Filled diamond	"Has-a", **strong** ownership (part cannot exist without the whole)	House has Rooms
Generalization	Hollow triangle	"Is-a"	Car is-a Vehicle
Dependency	Dashed open arrow	One class depends on another temporarily (e.g. method parameter)	OrderProcessor → PaymentGateway

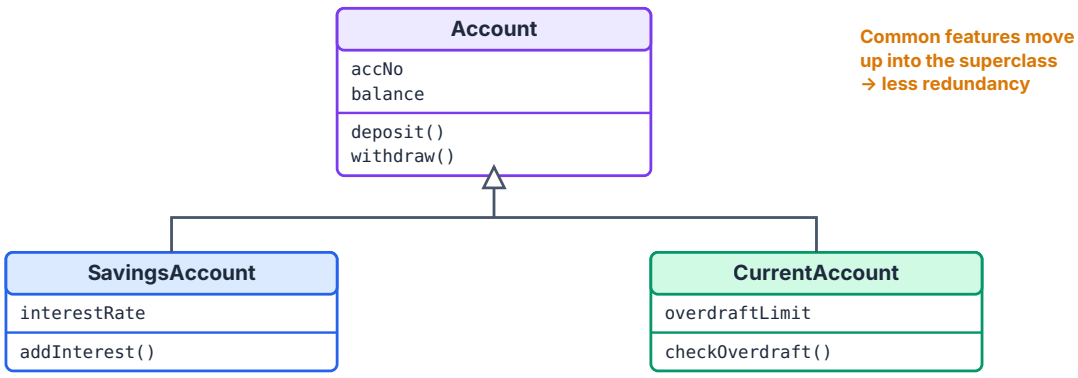


Fig. 48 — Generalization — common features move up into a superclass

🧠 **Yaad rakho**

Khali heera = **kamzor malik** (aggregation). Bhara heera = **pakka malik** (composition). Khula teer dotted = **kaam chalau** (dependency).

⭐ **Exam tip**

For a marks-scoring answer draw all five relationships in one figure like Fig. above and add one example each. Remember: **Association < Aggregation < Composition** in strength of ownership.

Object diagram

📘 **Definition**

An **object diagram** is an instance-level snapshot of a class diagram at a specific moment, showing actual objects, their current values and links.

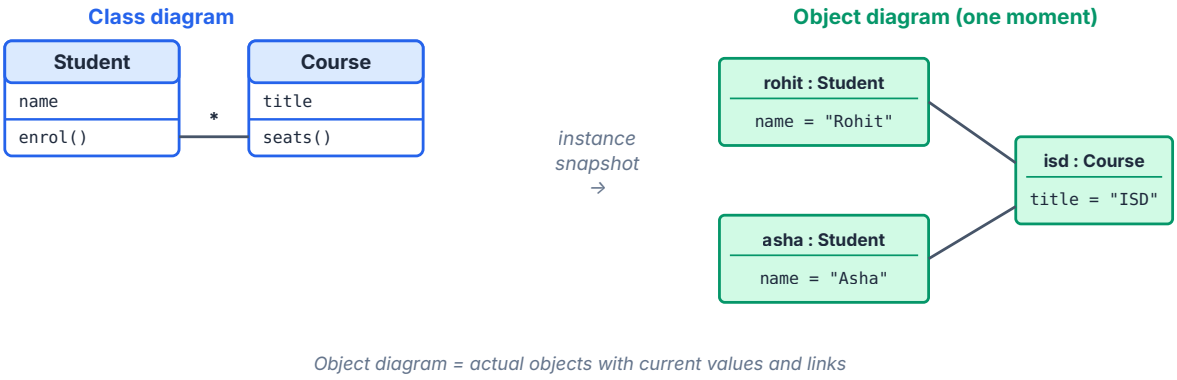


Fig. 49 — The same Student–Course classes shown as real objects at one moment

💡 **Why it matters**

Class diagram = the **blueprint**; object diagram = a **photo** of real objects built from that blueprint at one instant.

4.4 Sequence Diagrams

📘 Definition

A **sequence diagram** shows how objects interact over time in one scenario, emphasising the **order of messages**.

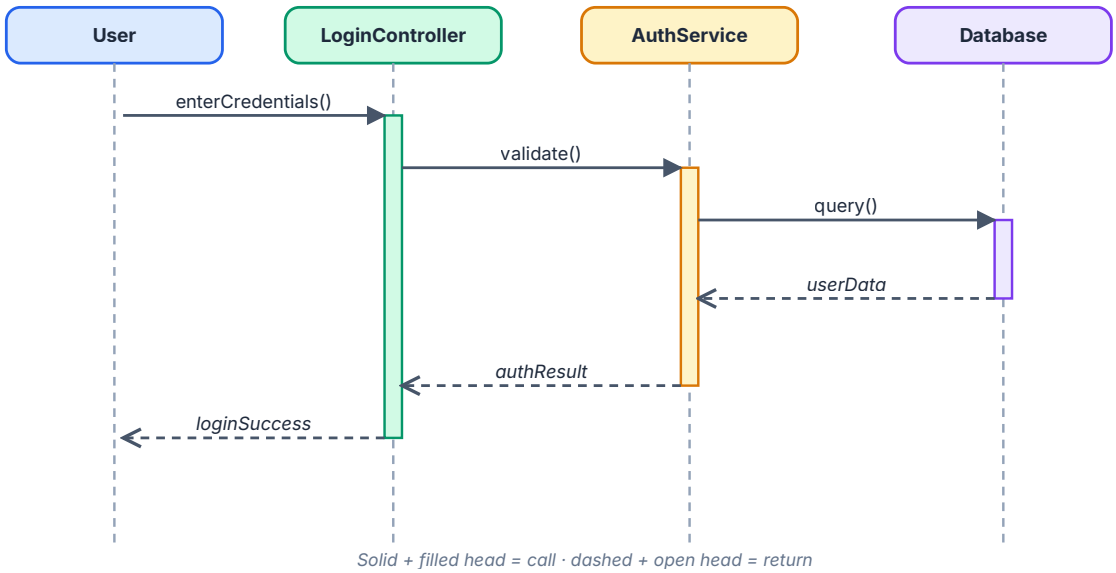


Fig. 50 — Login scenario — User, LoginController, AuthService and Database

Element	Meaning
Lifeline	Vertical dashed line — an object's existence over time
Activation bar	Thin rectangle on a lifeline — when the object is active
Message	Horizontal arrow between lifelines — a method call

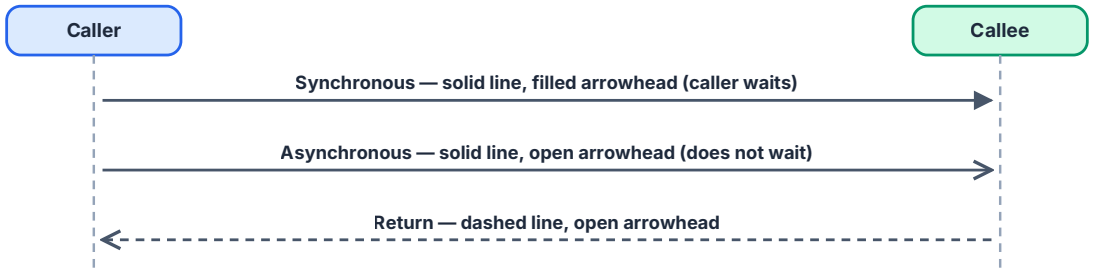


Fig. 51 — Three kinds of message arrows

Message	Notation
Synchronous	Solid line, filled arrowhead — caller waits for the response
Asynchronous	Solid line, open arrowhead — caller does not wait
Return	Dashed line, open arrowhead

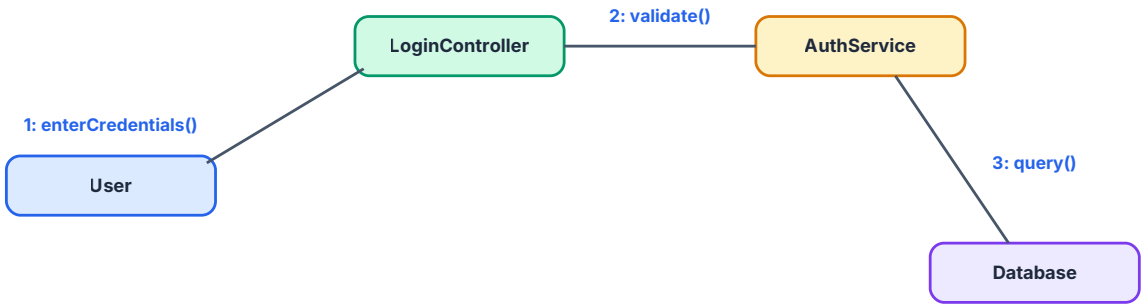
⭐ **Exam tip**

Draw lifelines first, then messages from top to bottom, then add activation bars. Show the **return** messages dashed — examiners check this.

4.5 Collaboration / Communication Diagrams

📘 **Definition**

A **collaboration diagram** shows the same information as a sequence diagram but emphasises the **relationships (links) between objects**. Objects are placed freely and messages are **numbered** (1, 2, 3...).



Same information as the sequence diagram — shows LINKS, order is the number

Fig. 52 — The same login scenario as a collaboration diagram — order shown by numbers

Sequence diagram	Collaboration diagram
Emphasises **time order** of messages	Emphasises **structural relationships** between objects
Objects arranged horizontally, lifelines vertical	Objects arranged freely, connected by links
Easy to see timing / duration	Easy to see how many objects one object is directly connected to
Order shown by vertical position	Order shown by sequence numbers on links

🧠 **Yaad rakho**

Sequence = **kab** (time). Collaboration = **kiske saath** (links).

4.6 State-Chart Diagrams (State Machine)

📘 **Definition**

A **state-chart diagram** shows the different states an object can be in during its lifetime and the events/ transitions that move it between states.

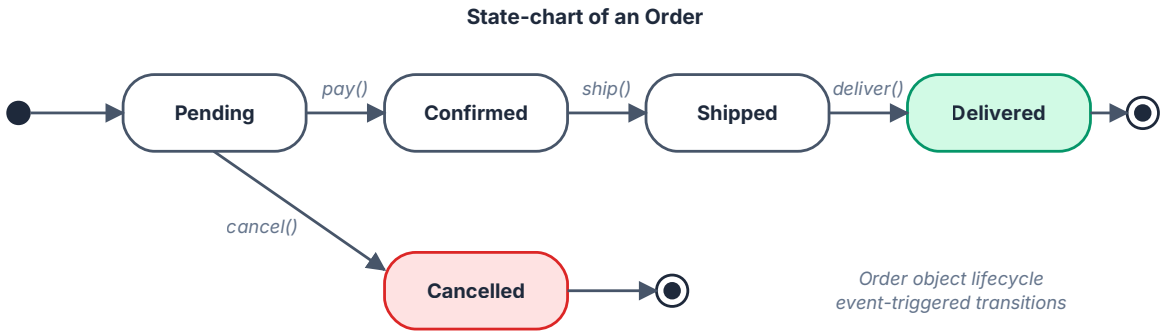


Fig. 53 — Order lifecycle — Pending → Confirmed → Shipped → Delivered, or Cancelled

Element	Notation
State	Rounded rectangle ("Pending", "Shipped")
Initial state	Filled black circle
Final state	Filled circle with a ring around it
Transition	Arrow labelled with the triggering event/condition

Useful for objects with complex, event-driven behaviour — e.g. an Order: Placed → Shipped → Delivered → Returned. The figure's cancel() branch from Pending follows your notes' example.

4.7 Activity Diagrams

Definition

An **activity diagram** models workflow / business-process logic — like a flowchart, with sequential and **parallel** activities, decisions and control flow.

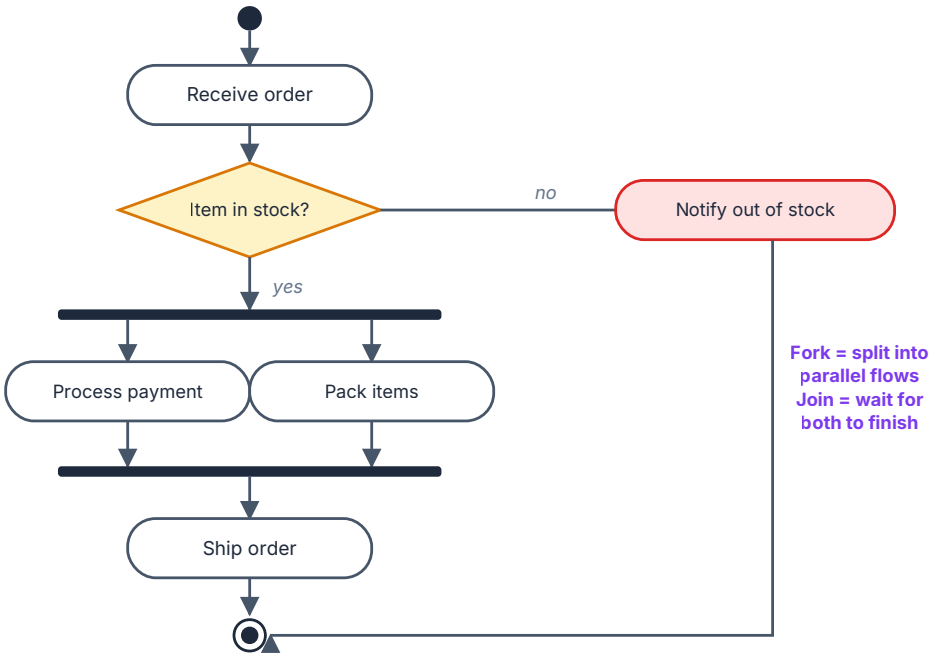


Fig. 54 — Order handling — a decision, then payment and packing in parallel

Element	Notation
Initial node	Filled black circle (start)
Activity / action	Rounded rectangle — a step/task
Decision node	Diamond — branch on a condition (like if-else)
Fork / Join	Thick bar — split into / merge from parallel activities
Final node	Filled circle with ring (end)

Activity diagram	State-chart diagram
Models **workflow / process** (verb-focused: actions)	Models **states of a single object** (noun-focused: conditions)
Good for business processes, algorithms	Good for the lifecycle of one object
Supports parallel flows (fork/join) natively	Focuses on event-triggered transitions between states

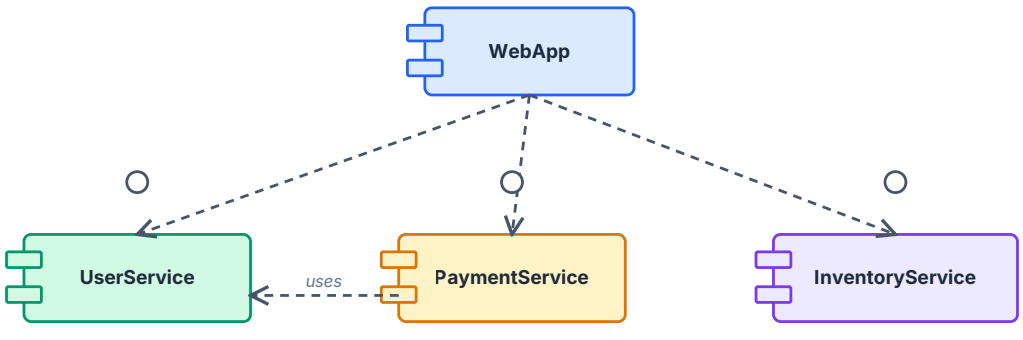
⚠️ **Careful**

Commonly confused pair! If the question talks about **steps / process** → activity diagram. If it talks about **states of one object over its life** → state-chart.

4.8 Component Diagrams

📘 **Definition**

A **component diagram** shows how the system is divided into physical/logical components (executables, libraries, packages, modules) and the dependencies/interfaces between them — the **static implementation view**.



Component diagram = LOGICAL pieces & dependencies (lollipop = interface offered)

Fig. 55 — E-commerce components — each exposes an interface that others depend on

🏭 **Real-life use case**

An e-commerce system: UserService, PaymentService and InventoryService each expose an interface; the web app depends on them. If PaymentService is replaced, only its interface contract must be kept.

4.9 Deployment Diagrams

📘 Definition

A **deployment diagram** shows the **physical architecture**: how software components are deployed on hardware nodes and how the nodes communicate. It answers "where does each part physically run?"



Node = 3-D box · Artifact = file (.war, image) · Path = network link

Answers: where does each part physically run?

Fig. 56 — Client, application server and database server, with their artefacts

Element	Meaning
Node	3-D box — physical hardware or an execution environment (container, VM)
Artifact	Physical file deployed on a node (.war file, Docker image)
Communication path	Line between nodes — a network connection

Component diagram	Deployment diagram
LOGICAL organisation of software modules	PHYSICAL deployment of components on hardware nodes
Focus: software architecture, dependencies	Focus: infrastructure, where software runs
"PaymentService depends on AuthService"	"PaymentService runs on Server A, AuthService on Server B"

🧠 Yaad rakho

Component = **software ka naksha** (logical). Deployment = **machine ka naksha** (physical).

4.10 Generalization, Domain Model Refinement, Architecture, Packaging

- **Generalization** — abstracting common features of several classes into a shared superclass (basis of inheritance); reduces redundancy.
- **Domain model refinement** — the initial conceptual domain model is progressively refined with attributes, methods, relationships and resolved ambiguities as analysis deepens into design.
- **Architecture (in OOAD)** — the organisation of classes/components into subsystems/layers, e.g. 3-tier: Presentation, Business Logic, Data.
- **Packaging model elements** — grouping related classes/components into **packages** (namespaces), shown in package diagrams as a folder-tab box.

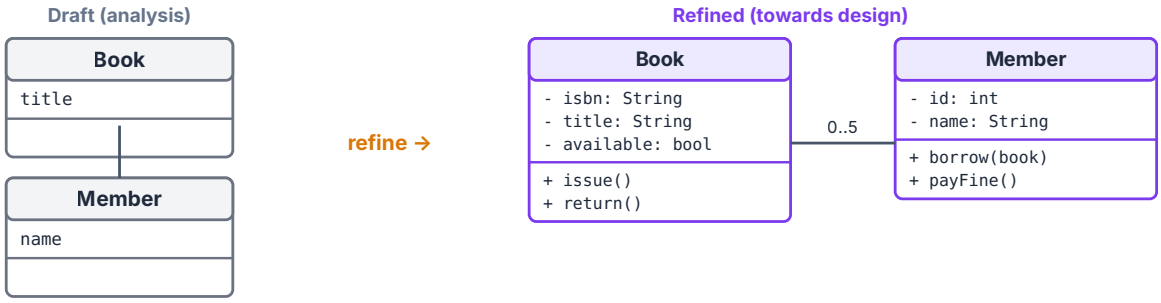


Fig. 57 — Domain model refinement — a draft grows attributes, methods and multiplicities

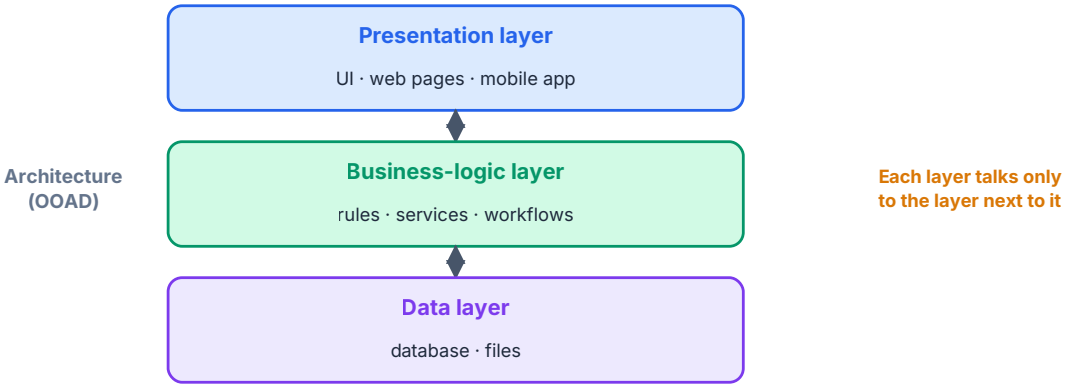


Fig. 58 — 3-tier architecture — presentation, business logic and data layers

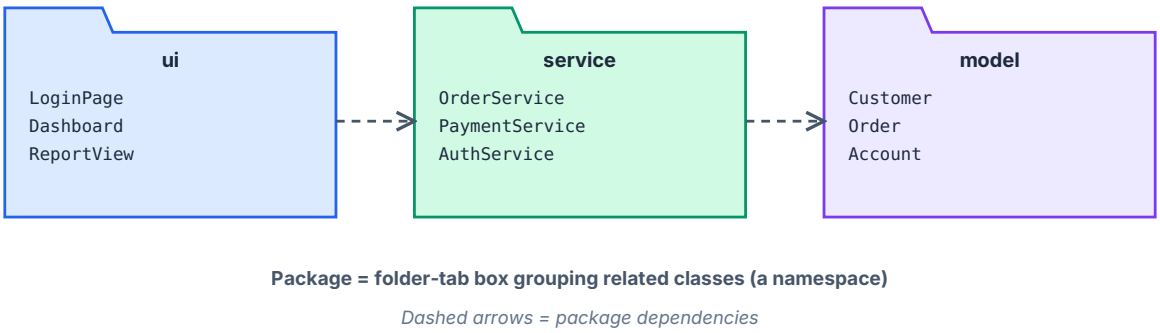


Fig. 59 — Package diagram — related classes grouped into ui, service and model packages

> ⚠️ **Careful**
>
> The class names, attributes and multiplicity (0..5) in the refinement figure are an illustration of the idea, not from your notes.

4.11 Case Study of Large-Scale Systems (exam approach)

📘 Definition

For a large-scale case study (e.g. **"Model an Online Banking System using UML"**) follow a fixed order of diagrams.

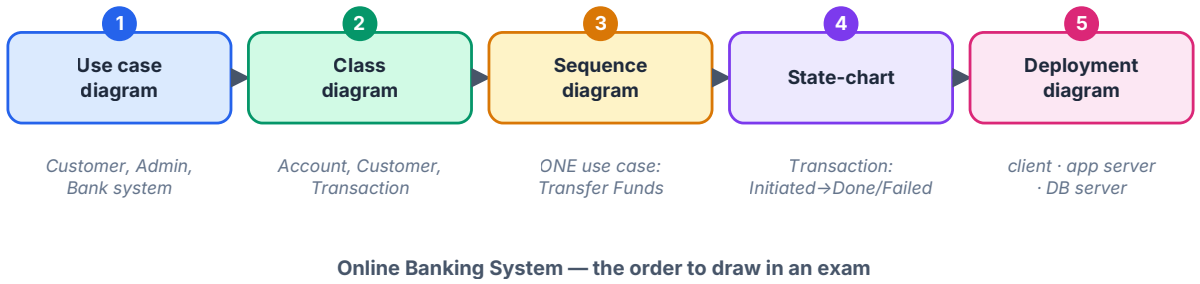


Fig. 60 — The order of UML diagrams to draw for a large-system case study

1. **Use case diagram** — actors (Customer, Admin, Bank System) and use cases (Login, Transfer Funds, View Statement).
2. **Class diagram** — key classes (Account, Customer, Transaction) and relationships.
3. **Sequence diagram** — pick **ONE** critical use case (e.g. Transfer Funds).
4. **State-chart** — if lifecycle is asked: Transaction Initiated → Processing → Completed/Failed.
5. **Deployment diagram** — if deployment is asked: client app, application server, database server.

⭐ Exam tip

Same structured mindset as the Unit I case-study method (Sec. 1.5), applied to UML. Do **not** draw every diagram for every system — pick the ones the question asks for.

❓ Likely exam questions

- What is UML? Classify UML diagrams.
- Draw a use case diagram for an online shop / library / ATM. Differentiate include, extend and generalization.
- Explain class-diagram relationships with notation and examples.
- Differentiate aggregation and composition.
- Draw a sequence diagram for login / transfer funds. Explain the message types.
- Sequence vs collaboration diagram.
- Draw a state-chart for an Order object.
- Activity diagram vs state-chart diagram.
- Component diagram vs deployment diagram.
- Model an Online Banking System using UML.

Unit IV references: Booch, Jacobson, Rumbaugh — *The UML User Guide* · OMG UML specification · Visual Paradigm and Lucidchart UML tutorials · practise drawing on draw.io / diagrams.net · GFG: UML diagrams · Use case diagram · Sequence diagram

UNIT V · CO4 / CO5

Implementation and Testing

TDD | Verification vs validation | Black / white / gray box | Testing levels | Integration approaches | Alpha & beta | Debugging | Automation | Tools

5.1 Traditional Implementation vs Test-Driven Development (TDD)

Definition

Test-Driven Development means writing a **failing test first**, then the minimum code to pass it, then refactoring.

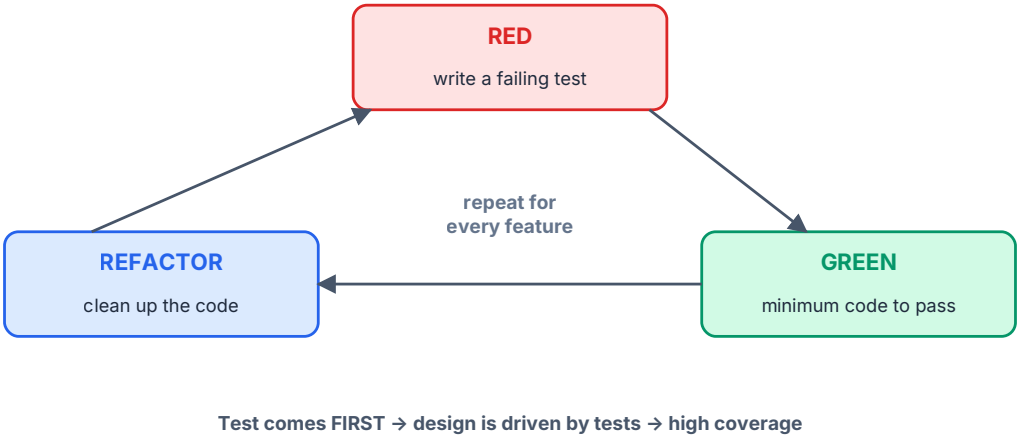


Fig. 61 — The TDD cycle — Red, Green, Refactor, repeat



Fig. 62 — Code-first vs test-first workflows

Traditional (code-first)	TDD
Write code first, then tests to verify it	Write a **failing** test first, then minimal code to pass it, then refactor
Testing is often an afterthought	Testing drives the design from the start
Cycle: Design → Code → Test	Cycle: **Red → Green → Refactor**
Can lead to lower test coverage	Naturally high coverage — every feature starts with a test

🏭 **Real-life use case — One TDD round (illustration, not from the notes)**

Requirement: "discount is 10% above ₹1000". **Red** — write a test that expects 1800 for a ₹2000 cart and watch it fail. **Green** — write the smallest `discount()` that passes. **Refactor** — tidy names and duplicates, keeping the test green.

```java
// RED: this test fails first (discount() does not exist yet)
@Test void tenPercentOffAbove1000() {
    assertEquals(1800.0, Cart.discount(2000.0), 0.01);
}
// GREEN: smallest code that makes it pass
static double discount(double amt) {
    return amt > 1000 ? amt * 0.9 : amt;
}
// REFACTOR: replace magic numbers with named constants, keep test green
```

🧠 **Yaad rakho**

Red → Green → Refactor: "Pehle fail karao, phir pass karao, phir saaf karo."

📖 GFG: Test Driven Development

5.2 Testing of Information Systems

📘 **Definition**

Testing verifies that the system meets requirements (**verification** — "are we building the product **right**?") and validates user needs (**validation** — "are we building the **right** product?").

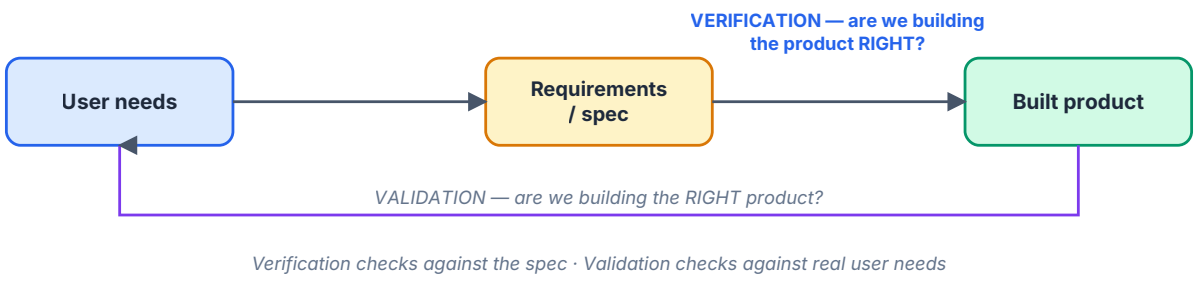


Fig. 63 — Verification checks against the spec; validation checks against what users need

> 🧠 **Yaad rakho**
>
> Verification = **sahi banaya?** (spec ke hisaab se). Validation = **sahi cheez banayi?** (user ki zaroorat ke hisaab se).

5.3 Testing Strategies

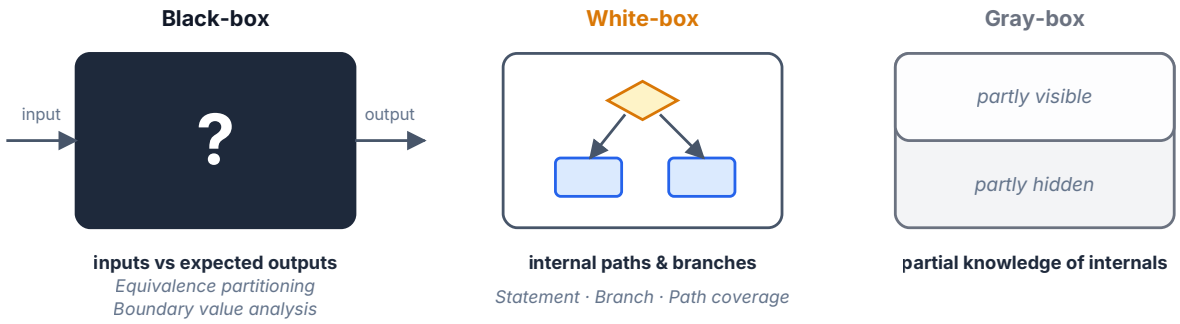


Fig. 64 — Black-box, white-box and gray-box testing

Strategy	What it tests	Techniques
Black-box	Functionality WITHOUT looking at internal code; inputs/ outputs vs specification	Equivalence partitioning, boundary value analysis
White-box	Internal code structure/logic (paths, branches, conditions)	Statement, branch, path coverage
Gray-box	Combination — tester has partial knowledge of internals	—

Black-box	White-box
Tester needs no coding knowledge	Tester needs programming knowledge
Focus: functionality vs requirements	Focus: internal logic / code paths
Done by testers, end-users	Done by developers
Example: login with valid/invalid credentials	Example: every if-else branch in the login function is executed

> ⭐ **Exam tip**
>
> In an answer, always name the techniques: **Equivalence Partitioning + Boundary Value Analysis** for black-box; **Statement / Branch / Path coverage** for white-box.

> 🧠 **Yaad rakho**
>
> Black = **andar nahi dikhta**. White = **andar ka code dikhta hai**. Gray = **thoda sa**.

5.4 Levels of Testing

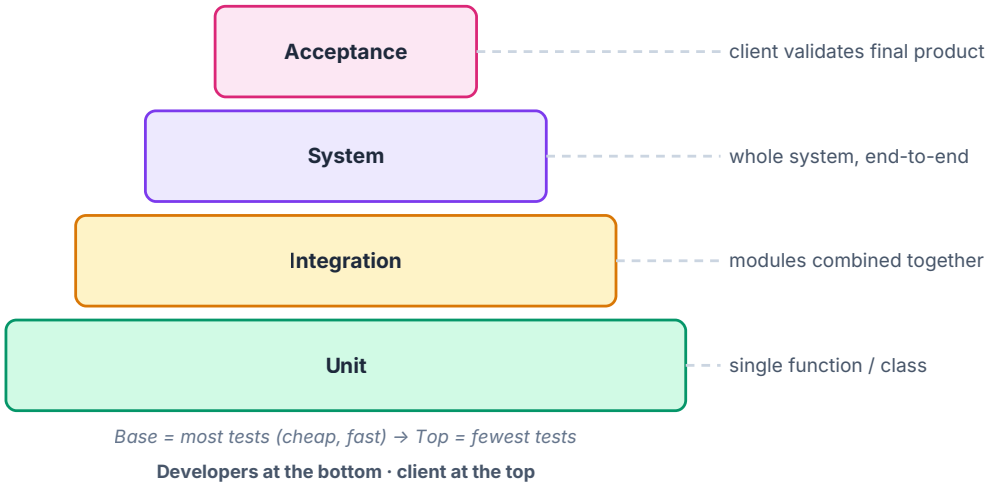


Fig. 65 — Testing pyramid — base has the most tests, the top has the fewest

1. **Unit testing** — individual functions/methods/classes in isolation; done by developers (JUnit, Jest).
2. **Integration testing** — interaction between combined modules.
3. **System testing** — the complete integrated system against overall requirements (functional + non-functional).
4. **Acceptance testing** — final validation, often by the client/end-user, before go-live.

Integration approaches

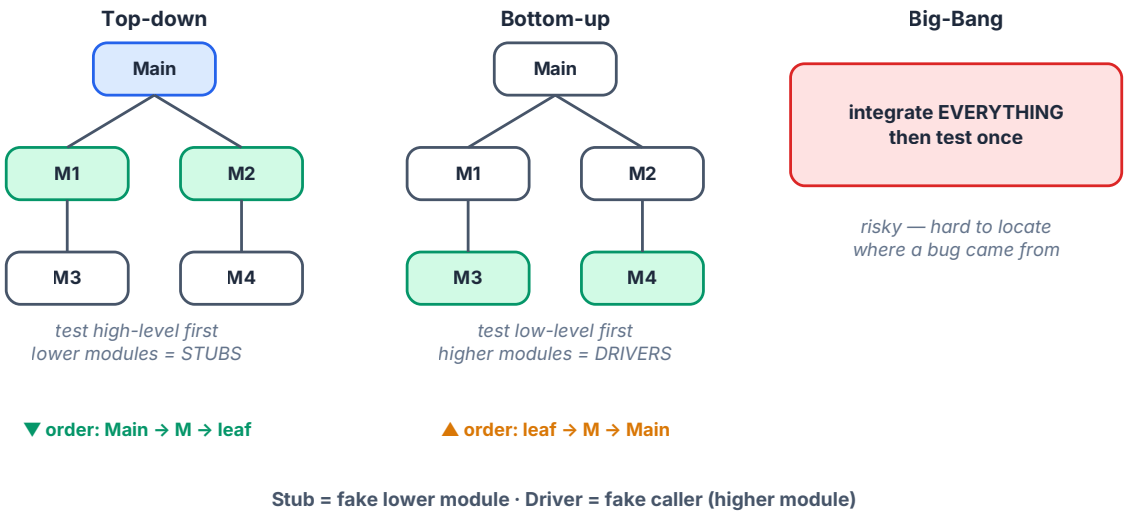


Fig. 66 — Top-down uses stubs, bottom-up uses drivers, big-bang integrates everything at once

Approach	How
Top-down	Test high-level modules first, using **stubs** for lower modules
Bottom-up	Test low-level modules first, using **drivers** to simulate higher modules
Big-Bang	Integrate everything at once, then test — **risky**

Alpha and beta testing

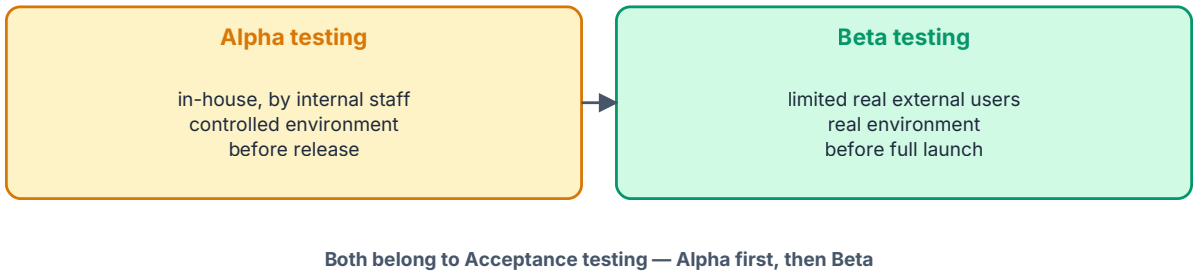


Fig. 67 — Alpha is in-house; beta is with limited real external users

Acceptance testing includes **Alpha testing** (in-house, by internal staff) and **Beta testing** (by a limited set of real external users in a real environment).

🏭 Real-life use case

A new banking app is first used by the bank's own employees (alpha), then released to a few thousand volunteer customers (beta) before the full launch.

⚠️ Careful

Stub = a fake **lower** module (used top-down). Driver = a fake **caller/higher** module (used bottom-up). Do not swap them.

🧠 Yaad rakho

Unit → Integration → System → Acceptance: "**U-I-S-A**". Stub neeche ka nakli, Driver upar ka nakli.

5.5 Debugging

📘 Definition

Debugging is finding and fixing the **root cause** of a defect once testing has revealed a failure. **Testing finds bugs; debugging finds the cause and fixes it.**

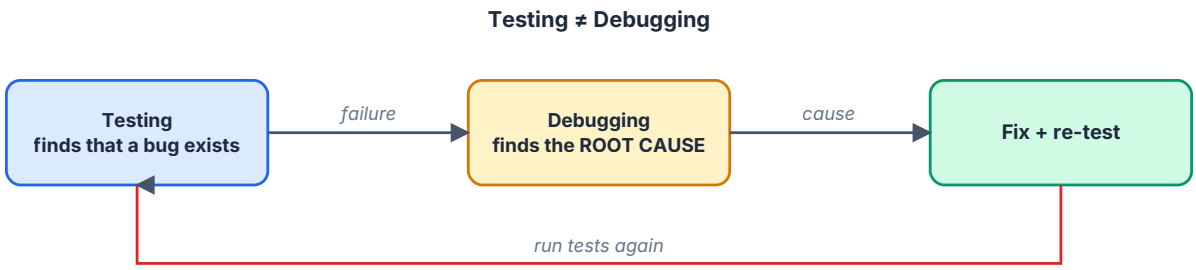


Fig. 68 — Testing finds the failure, debugging finds and fixes the cause

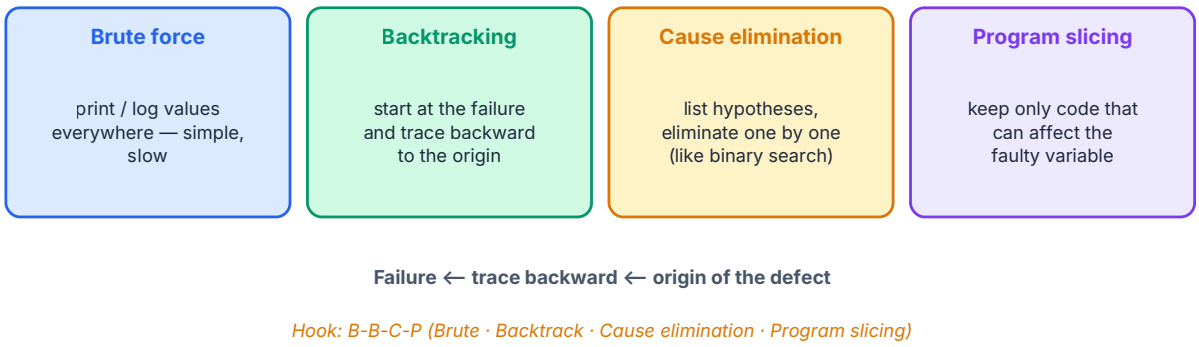


Fig. 69 — Four debugging approaches

Approach	Idea
Brute force	Add print/log statements everywhere to trace values — simple but inefficient
Backtracking	Start at the failure point and trace backward to the origin
Cause elimination	Form hypotheses about causes and eliminate them systematically (like binary search on causes)
Program slicing	Isolate only the code that can affect a specific variable/output to narrow the search

🧠 **Yaad rakho**

Debug ke 4 tareeke: **B-B-C-P** — Brute, Backtrack, Cause elimination, Program slicing.

5.6 Automation Testing

📘 **Definition**

Automation testing uses tools/scripts to execute test cases automatically instead of manually — especially valuable for repetitive tests such as **regression testing** after every code change.

Point	Manual testing	Automation testing
Executed by	A human	Scripts / tools
Best for	Exploratory / usability testing	Repetitive regression tests (faster)
Cost	Higher long-term (repeated effort)	Higher upfront (script development), lower long-term
Catches	Unexpected / visual issues humans notice	Only what is explicitly scripted

🏭 **Real-life use case**

An e-commerce site runs hundreds of Selenium checks on every release overnight (automation), while a person tries the new checkout page by hand to judge how it *feels* (manual).

5.7 Software Testing Tools

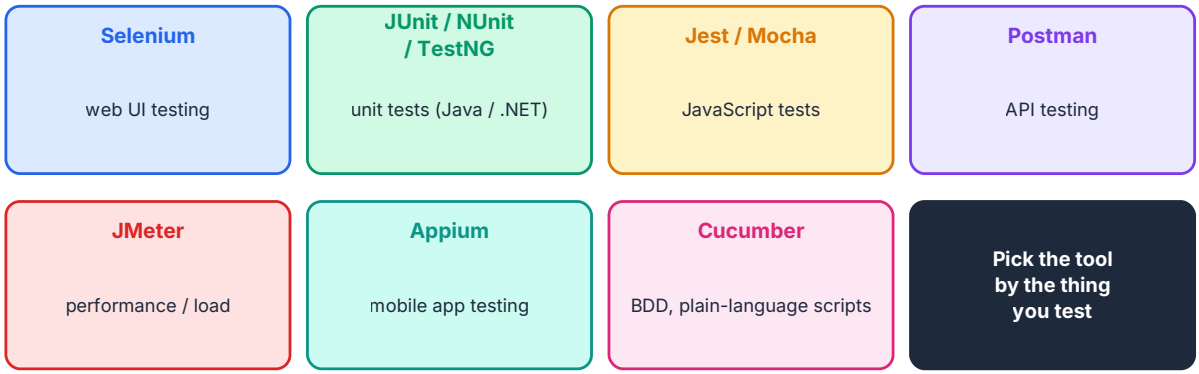


Fig. 70 — Common testing tools and what each is used for

Tool	Purpose
Selenium	Automated web-application UI testing
JUnit / NUnit / TestNG	Unit-testing frameworks (Java / .NET)
Jest / Mocha	JavaScript unit/integration testing
Postman	API testing
JMeter	Performance / load testing
Appium	Mobile application testing
Cucumber	Behaviour-Driven Development (BDD) with natural-language test scripts

🧠 **Yaad rakho**

Selenium = **web**, Postman = **API**, JMeter = **load**, Appium = **mobile**, Cucumber = **BDD**, JUnit = **unit**.

❓ **Likely exam questions**

- Compare traditional development with TDD. Draw the TDD cycle.
- Verification vs validation.
- Explain black-box, white-box and gray-box testing with techniques.
- Explain the levels of testing with a pyramid.
- Explain top-down, bottom-up and big-bang integration. What are stubs and drivers?
- Alpha vs beta testing.
- Testing vs debugging. Explain the debugging approaches.
- Manual vs automation testing. Name five testing tools and their purpose.

Unit V references: Kent Beck, *Test-Driven Development: By Example* · Glenford Myers, *The Art of Software Testing* · Selenium official documentation · Martin Fowler on TDD · GFG: Software testing · Levels of testing

Quick comparison sheet

One-glance tables for the last day before the exam. Each row is a favourite "differentiate between" question.

Units I–II

Topic	Remember
IS / IT	People + process + technology / just the technology part
Waterfall / CBD / Agile / RUP	Sequential, stable reqs / reuse components / sprints, change / 4 phases I-E-C-T, large systems
Iron triangle	Scope-Cost-Time, Quality in the centre
Chief / Democratic / Matrix team	Fast, low overhead / slow, high overhead / medium, medium
FP / LOC	Language-independent, early, complex / language-dependent, late, simple
COCOMO	Effort = a × KLOC^b — Organic / Semi-detached / Embedded
Brooks's Law	Adding people to a late project makes it later
Gantt / PERT-CPM	Timeline bars / dependency network + critical path
SCM	Identification, version control, change control, status accounting, audits
Risk cycle	Identify → Analyse → Prioritise → Mitigate → Monitor

Units III–V

Topic	Remember
RE cycle	Elicitation → Analysis → Specification → Validation (+ Management)
Structured / OO analysis	DFD, process-first / UML, object-first
Throwaway / evolutionary prototype	Discarded / refined into the final system
Functional / non-functional	WHAT it does / HOW WELL
Coupling / cohesion	LOW between modules / HIGH inside a module
Refactoring	Change structure, NOT behaviour
Include / extend	Mandatory / optional (arrow points to base)
Aggregation / composition	Hollow diamond, weak / filled diamond, strong
Sequence / collaboration	Time order / links with numbers
Activity / state-chart	Workflow / one object's states
Component / deployment	Logical software / physical hardware
TDD cycle	Red → Green → Refactor
Black / white box	Functionality, testers / code paths, developers
Testing pyramid	Unit → Integration → System → Acceptance
Top-down / bottom-up	Stubs / drivers
Testing / debugging	Finds bugs / finds root cause and fixes

Exam-day revision checklist

Tick these off — if you can **write or draw** each one from memory, you are well covered.

- [ ] IS definition, IS vs IT, input-process-output-feedback
- [ ] Eight challenges in ISD
- [ ] Six elements of an IS and eight roles
- [ ] Case-study approach — five steps
- [ ] Waterfall, CBD, Agile (4 values), RUP (4 phases)
- [ ] Process-model comparison table
- [ ] Iron triangle diagram with quality in the middle
- [ ] Reasons for project success and failure; CHAOS Report
- [ ] Project life cycle — five phases
- [ ] Three team structures compared
- [ ] Function Points vs LOC
- [ ] COCOMO formula and three project types
- [ ] Brooks's Law with reason
- [ ] Gantt vs PERT/CPM and the critical path
- [ ] Five SCM activities
- [ ] Risk-management cycle and mitigation options
- [ ] RE cycle and elicitation techniques
- [ ] Structured vs OO analysis; throwaway vs evolutionary prototype
- [ ] Six SRS qualities; functional vs non-functional
- [ ] Seven design fundamentals — low coupling, high cohesion
- [ ] Design process: architecture → interface → component → data
- [ ] Refactoring and its techniques
- [ ] UML classification — 5 structural + 5 behavioral
- [ ] Use case: include / extend / generalization
- [ ] Class diagram — five relationships with notation
- [ ] Sequence diagram with message types
- [ ] State-chart and activity diagram elements
- [ ] Component vs deployment diagram
- [ ] UML case-study order for a banking system
- [ ] TDD cycle; verification vs validation
- [ ] Black / white / gray box and their techniques
- [ ] Testing levels, stubs vs drivers, alpha vs beta
- [ ] Debugging approaches; testing tools and purposes

Recommended resources

Resource	Covers
Pressman — *Software Engineering: A Practitioner's Approach*	Units I–III (process models, project management, COCOMO, risk)
Sommerville — *Software Engineering*	Units I and III (processes, requirements, design)
Fowler — *Refactoring*	Unit III
Booch, Jacobson, Rumbaugh — *UML User Guide* · omg.org/spec/UML	Unit IV
draw.io / diagrams.net — free tool	Practise drawing UML
Beck — *TDD: By Example* · Myers — *The Art of Software Testing*	Unit V
agilemanifesto.org · Selenium docs	Agile values · test automation
geeksforgeeks.org/software-engineering	All units, with extra practice

All diagrams in this book were drawn for this book. Per-topic "GFG" links open GeeksforGeeks **search** pages (as in your notes, they are not single fixed articles) for extra reading.